知能工学実験 II

Python による人工知能プログラミング実験

広島市立大学情報科学部知能工学科編

2026-06-01

本書は、広島市立大学情報科学部知能工学科の「知能工学実験II」（人工知能プログラミング実験）の教材として作成されたものである。Python によるオセロゲームの実装を通じて、ゲーム木探索、評価関数の設計、テスト駆動開発、バージョン管理などの人工知能プログラミングとシステム開発の基礎を学ぶことを目的とする。本資料は 2025 年度まで使用された人工知能プログラミング実験のテキスト（LISP 版）をもとに、2026 年度の内容に合わせて改訂したものであり、説明の整理と最新の Python の機能を活用したコード例の更新を行っている。LISP から Python への移行には ChatGPT を補助的に活用した。

Table of contents

第 1 章　はじめに 7
- 1.1 本実験の特徴 7
- 1.2 学習する内容 7
- 1.3 完成像 8
- 1.4 本書の使い方 9

第 2 章　人工知能の歴史 11
- 2.1 人工知能という概念の誕生 11
- 2.2 第一次 AI ブーム：推論と探索 12
- 2.3 第二次 AI ブーム：知識表現とエキスパートシステム 12
- 2.4 第三次 AI ブーム：機械学習とニューラルネットワーク 12
- 2.5 現在：基盤モデルと生成 AI 13
- 2.6 本実験の位置づけ 13

第 3 章　環境構築と準備 15
- 3.1 Visual Studio Code 15
- 3.2 Docker 16
- 3.3 VSCode から Docker コンテナを起動する 17
- 3.4 まとめ 17

第 4 章　Git によるバージョン管理 19
- 4.1 バージョン管理とは 19
- 4.2 Git の準備 20
- 4.3 Git の基本概念 20
- 4.4 Git の基本コマンド 21
- 4.5 練習：実際に commit してみる 22
- 4.6 推奨されるコミットの粒度 23
- 4.7 まとめ 24

第 5 章　`pytest` によるテスト駆動 25
- 5.1 `pytest` のインストールと実行 25

5.2 はじめてのテスト：assert 文 26
5.3 pytest 形式のテスト 26
5.4 失敗するテストの読み方 27
5.5 オセロのテスト例（先取り） 27
5.6 開発の流れ 28
5.7 まとめ 29

第 6 章 ゲームと探索 31
6.1 ゲームとは 31
6.2 ゲームの分類 31
6.3 ゲームの解析 32
6.4 状態の表現と探索木 32
6.5 minimax 法 33
6.6 α β 枝刈り（先取り） 35
6.7 評価値（コスト）の計算 37
6.8 まとめ 37

第 7 章 オセロゲームの基本実装 39
7.1 本章の到達目標 39
7.2 オセロゲームのルール 39
7.3 盤面の表現 40
7.4 手番と表示 42
7.5 盤面の走査 43
7.6 盤面の更新 44
7.7 終局と評価 45
7.8 人間による着手 47
7.9 人間対人間のゲーム 47
7.10 まとめ 49

第 8 章 minimax 法によるゲーム木探索 51
8.1 節点のデータ構造 51
8.2 節点の操作関数 52
8.3 minimax 法の実装 53
8.4 コンピュータの着手 55
8.5 人間対コンピュータ、コンピュータ対コンピュータ 55
8.6 まとめ 56

第 9 章 α β 枝刈り 57
9.1 α と β 57
9.2 α β 法の本体 57

9.3 子節点の探索 58
9.4 コンピュータの着手に組み込む 59
9.5 探索効率の比較 60
9.6 まとめ 61

第 10 章 ヒューリスティック評価関数 63
10.1 局面の段階 63
10.2 位置による評価（board_eval2） 64
10.3 可動性の評価（board_eval3） 65
10.4 確定石の評価（board_eval4） 66
10.5 X 打ち・C 打ちの評価（board_eval5） 68
10.6 評価関数を組み合わせる 69
10.7 次章以降への接続 70
10.8 まとめ 70

第 11 章 評価関数の改良と最終課題 71
11.1 改良の方針 71
11.2 実験の進め方 71
11.3 ルール部分のテスト 72
11.4 最終課題 73
11.5 改良アイデアの種 74

第 12 章 メタヒューリスティクスによる評価関数の最適化 75
12.1 本章の到達目標 75
12.2 問題の定式化 76
12.3 共通フレームワーク 76
12.4 山登り法（Hill Climbing） 78
12.5 焼きなまし法（Simulated Annealing） 80
12.6 遺伝的アルゴリズム（Genetic Algorithm, GA） 81
12.7 アルゴリズム比較実験 83
12.8 発展：他のアルゴリズム 84
12.9 まとめ 85

第 13 章 発展：機械学習による評価関数の構築 87
13.1 本章の到達目標 87
13.2 学習用データの準備 88
13.3 線形回帰による評価関数 89
13.4 ロジスティック回帰 91
13.5 小規模ニューラルネットワーク 91
13.6 学習データの注意点 92

13.7 発展：強化学習 92
13.8 レポートに含める内容（学習を選択した場合） 93
13.9 まとめ 93

第 14 章 対戦実験と GUI 95
14.1 CUI での実行 95
14.2 対戦ログの保存 96
14.3 条件を変えた比較 96
14.4 GUI による対戦：`reversi-gui.py` 96
14.5 Git で実験を管理する 97
14.6 提出前の確認 98
14.7 発展：外部対戦サーバとの接続 98
14.8 まとめ 98

第 15 章 まとめと評価 99
15.1 提出物 99
15.2 レポートの観点 99
15.3 口頭説明 100
15.4 評価の重み（目安） 100
15.5 学びの振り返り 101

第 16 章 参考文献 103

第 1 章

はじめに

人工知能の研究は、推論、探索、知識表現、機械学習、深層学習、生成 AI へと発展してきた。近年の生成 AI の著しい性能向上は、大規模なデータと計算資源、そして適切なアルゴリズムが組み合わさった結果である。このような華やかな成果の裏側には、人工知能の基礎的な概念である「**状態をどのように表現するか**」「**限られた計算資源の中でどのように解を探索するか**」という古典的な問いがある。これらは、現在の機械学習や AI エージェントにも共通する考え方であり、人工知能を学ぶ上で避けて通れないテーマである。

本実験では、Python を用いて**オセロゲームを段階的に実装する**ことを通して、これらの基礎を体験的に学ぶ。

1.1 本実験の特徴

旧教材（2025 年度まで）ではプログラミング言語として LISP を用いていた。LISP は記号処理に特化した古典的な人工知能言語であり、ゲーム木探索や知識表現の研究で長く使われてきた。しかし、近年の機械学習・深層学習・生成 AI 研究は、ライブラリと開発エコシステムの豊かさから Python が事実上の標準言語となっている。そこで 2026 年度からは、本実験も Python による実装に切り替える。あわせて、現代のソフトウェア開発で必須となっている

- **Git によるバージョン管理**
- **`pytest` を用いた単体テスト**
- **VSCode + Dev Container による再現可能な開発環境**

を実験のフレームワークとして組み込む。

1.2 学習する内容

章	内容
1	人工知能の歴史
2	実験環境の構築（VSCode + Dev Container）
3	Git によるバージョン管理
4	`pytest` によるテスト駆動開発の基礎
5	ゲーム理論とゲーム木
6	オセロゲームの基本実装
7	minimax 法によるゲーム木探索
8	α β 枝刈り
9	ヒューリスティック評価関数
10	評価関数の改良と実験
11	メタヒューリスティクスによる評価関数の最適化（山登り法・焼きなまし法・GA）
12	発展：機械学習による評価関数の構築
13	対戦実験と GUI
14	最終課題と評価

これらは大きく次の 3 つの段階に分かれている。

1. **実装の基礎**（1〜8 章）：環境構築、ルール実装、minimax/α β までを完成させる。
2. **評価関数の設計**（9〜10 章）：人間の知識を評価関数として記述し、手動で改良する。
3. **自動化の階層**（11〜12 章）：人間の手を離れて、最適化アルゴリズムや機械学習で評価関数を改良する。

11 章と 12 章は、AI 研究の歴史でいえば「**人間の知識を作り込む**段階」から「**最適化が知識を獲得する**段階」、そして「**データから学ぶ**段階」への移行に対応している。これは、本書の第 1 章で振り返る AI の歴史的発展（推論 → 知識ベース →機械学習）の縮図でもある。

1.3　完成像

最終的には、`reversi_yourid.py`[*1]という 1 つの Python ファイルとして、

We know from *the first fundamental theorem of calculus* that for x in $[a, b]$:

$$\frac{d}{dx}\left(\int_a^x f(u)\,du\right) = f(x).$$

[*1] `yourid` は自分の ID に置き換えること。24xxx など。

- 人間 vs 人間
- 人間 vs コンピュータ（minimax / α β）
- コンピュータ vs コンピュータ

の三通りのオセロゲームを実行できるプログラムを完成させる。さらに、自分が設計した評価関数の特徴を**実験データに基づいて**説明できるようになることを目指す。

1.4　本書の使い方

各章には**例題**と**練習**がある。例題には基準となる実装例が示されているので、まずは写経し、動作を確認してから先に進む。練習は、その章で学んだ関数を自分の手で実装する課題である。どの練習にも動作確認用の `assert` 文がついているので、これが通ることを確認しながら進めること。こまめに `git commit` し、テストが通った状態を残すこと。

第 2 章

人工知能の歴史

人工知能（Artificial Intelligence: AI）研究の流れを駆け足で振り返ることで、本実験で扱うゲーム木探索や評価関数といった古典的な手法が、AI 全体のなかでどのような位置を占めているのかを理解する。

2.1 人工知能という概念の誕生

2.1.1 サイバネティクス

1940 年代、Norbert Wiener は『サイバネティクス（*Cybernetics: Or Control and Communication in the Animal and the Machine*, 1948）』において、生物と機械を共通の枠組みで論じた。ここでは**フィードバック制御**、**情報**、**通信**を共通の言葉として、神経系や社会、機械システムを統一的に説明することが試みられた。人工知能という言葉が登場する以前の段階で、「機械が知的にふるまうとはどういうことか」という問いに対する基礎が作られていた。

2.1.2 Turing テスト

1950 年、Alan Turing は論文 *Computing Machinery and Intelligence* で「機械は考えることができるか」という問いを設定した。答えにくいこの問いを、「ある質問者が、隔てた相手と文字でやりとりするとき、相手が人間か機械か区別できないなら、その機械は知的とみなしてよい」という**模倣ゲーム（imitation game）**に置き換えた。これが今日**チューリング・テスト**と呼ばれる思考実験である。Turing テストは、現代の対話型 AI の評価にも引き合いに出されることが多い。

2.1.3　ダートマス会議と AI の正式な誕生

1956 年、アメリカ・ダートマス大学で開催されたダートマス会議は、人工知能という概念が**正式に誕生した瞬間**とされている。この会議では、John McCarthy、Marvin Minsky、Allen Newell、Herbert Simon などの研究者が集まり、人工知能の研究の方向性や目標を議論した。ここで McCarthy が「Artificial Intelligence」という用語を提唱し、これ以降この語が学術用語として定着した。McCarthy は同時期にプログラミング言語 **LISP** を設計しており、その後数十年にわたって人工知能研究の主要言語となった。

2.2　第一次 AI ブーム：推論と探索

「知能とは論理的に物事を判断できることである」という見方のもと、**推論（reasoning）**と**探索（search）**によってさまざまな問題を解こうとする研究が盛んに行われた。迷路、パズル、定理証明、チェスや将棋などのゲームが題材となった。本実験で扱う**ゲーム木探索**、**minimax 法**、**α β 枝刈り**はまさにこの時期に体系化された手法であり、現代でもゲーム AI や経路計画の基礎となっている。

ただし、現実世界の問題は探索空間が膨大であり、また「常識」のような明文化されていない知識が必要になることも多い。これにより、第一次 AI ブームは 1970 年代に停滞期（AI の冬）を迎えた。

2.3　第二次 AI ブーム：知識表現とエキスパートシステム

第一次ブームの停滞は、**知能は知識なしには成り立たない**という認識をもたらした。そこで知識を計算機に明示的に蓄え、推論で利用する**エキスパートシステム**が研究の中心となった。医療診断（MYCIN）や化学解析（DENDRAL）、設定支援（XCON）などの実用例も登場した。

しかし、知識を人手で作り込むコストの高さ、知識の保守の難しさ、例外への対応の弱さといった限界が明らかになり、再び停滞期を迎える。

2.4　第三次 AI ブーム：機械学習とニューラルネットワーク

1990 年代以降、ハードウェアの性能向上に伴い大規模なデータセットの利用が現実的になった。同時に、自由度の高いニューラルネットワークの計算に必要な処理速度が向上し、機械学習の研究が再び盛んになった。

決定的な転機は 2012 年の ImageNet コンペティションである。G. Hinton らのグループ

が **AlexNet** という畳み込みニューラルネットワークを用いて、従来の画像認識手法を大幅に上回る性能を達成した。これを契機に**深層学習（Deep Learning）**が爆発的に普及し、画像認識、音声認識、自然言語処理、強化学習などの広い分野で応用されるようになった。

オセロやチェス、囲碁の世界でも、2016 年の AlphaGo が深層学習と探索を組み合わせることでトップ棋士に勝利し、ゲーム AI 研究の新たな段階を切り開いた。

2.5　現在：基盤モデルと生成 AI

2020 年代に入り、Transformer に基づく**大規模言語モデル**（Large Language Models, LLM）が登場した。ChatGPT に代表される対話型 AI は、文章生成、要約、コード生成、検索など多岐にわたる用途で日常的に利用されるようになっている。これらの**基盤モデル**は、機械学習・深層学習の延長線上にあるが、極めて大規模なモデルとデータを活用することで質的に新しい振る舞いを示している。

2.6　本実験の位置づけ

このように、AI 研究は「論理的な推論と探索」から「知識表現」、そして「データ駆動の学習」へと重心が移ってきた。しかし、現代のゲーム AI や強化学習エージェントの内部でも、**ゲーム木探索や評価関数の考え方は依然として重要な部品**として使われている。本実験で扱う minimax 法、α β 枝刈り、評価関数の設計は、AI の古典的な基礎であると同時に、現代 AI につながる「状態と探索」の感覚を養うための練習である。

第 3 章

環境構築と準備

本実験で用いるのは、これまでの講義・演習でも扱ってきた **Python** である。ただしオンラインサービスである Google Colaboratory ではなく、**ローカル環境に Python をインストールして実験を行う**。2026 年度は、知能工学実験室に備え付けられているデスクトップ PC を用いて実験を進める。

本学では BYOD（Bring Your Own Device）を推進しており、学生は個人のノート PC を持ち学習を進めている。しかし学生ごとに OS や環境が異なるため、本実験のように環境依存性の強い実験では、共通化が必要になる。そこで、統一された環境を提供するため **VSCode + Dev Containers + Docker** による仮想環境を導入する。

手順の概要は次の通りである。

1. VSCode をインストールする。
2. VSCode に Dev Containers 拡張機能をインストールする。
3. Docker をインストールする。
4. 配布される `lab_env.zip` を展開して VSCode で開き、Dev Containers でコンテナを起動する。
5. コンテナ内のターミナルで `python --version` を確認する。

3.1 Visual Studio Code

Visual Studio Code（VSCode）は Microsoft が提供する無料のコードエディタである。本実験では Python の編集、実行、デバッグ、テスト、Git 操作を VSCode 上で完結させる。

3.1.1 VSCode のインストール

次の URL からインストーラを取得する。

- Visual Studio Code

既にインストールされているものはそのまま使ってもよいが、最新版を利用すること。

3.1.2 Dev Containers のインストール

VSCode の拡張機能 Dev Containers をインストールする。これは VSCode から Docker コンテナを起動して、コンテナ内でコードを編集・実行できるようにする拡張機能である。

3.1.3 Python 拡張機能のインストール

合わせて、VSCode の拡張機能 Python をインストールしておく。コード補完、デバッガ、テストランナーなどがすぐに使えるようになる。

3.2 Docker

Docker はコンテナ型の仮想化ソフトウェアであり、OS の違いを吸収して同一の環境を提供することができる。VSCode の Dev Containers は Docker コンテナを起動するための拡張機能であり、Docker 自体は別途インストールする必要がある。

3.2.1 Docker のインストール

次の URL からインストーラを取得する。

- Docker

インストールが済んだら Docker を起動し、ターミナルで次のコマンドを実行してインストールを確認する。

```
docker --version
```

3.2.2 Docker イメージの取得

最後の準備として、本実験用に準備した開発コンテナ設定を取得する。UNIPA から `lab_env.zip` をダウンロードして、適当な場所に展開する。展開すると次のような構成になっている。

```
lab_env/
├── .devcontainer/
│   ├── devcontainer.json
│   └── Dockerfile
├── reversi_yourid.py     # 各自のプログラム
├── test_reversi.py       # テストファイル
└── pyproject.toml
```

`devcontainer.json` には Python のバージョン、必要な拡張機能、必要なパッケージ（`pytest`, `pygame` など）が指定されている。`Dockerfile` は Python ベースのイメージに対し、本実験で使うパッケージを上書きインストールする。これらのファイルは原則編集しないこと。

3.3 VSCode から Docker コンテナを起動する

VSCode を起動し、`lab_env` フォルダを開く。左下の緑色のアイコンをクリックし、**「コンテナーで再度開く（Reopen in Container）」**を選択する。これにより Docker コンテナが起動し、VSCode がコンテナ内で再起動される。

> ⚠ Warning
>
> 最初の起動はイメージのビルドが行われるため**非常に時間がかかる**。授業時間のロスを避けるため、次回以降の実験では授業前にあらかじめ起動しておくこと。

コンテナ内で VSCode が起動したら、ターミナルを開いて、次のコマンドで Python の実行環境が構築されていることを確認する。

```
python --version
pip list | grep -E "pytest|pygame"
```

`Python 3.x.x` と表示され、`pytest` と `pygame` がインストールされていれば成功である。

3.4 まとめ

- 本実験では VSCode + Dev Containers + Docker により、統一された Python 実行環境を用いる。
- 配布される `lab_env.zip` には Dev Container 設定が含まれており、VSCode で開いて「コンテナーで再度開く」を選ぶだけで環境が立ち上がる。
- 初回のビルドは時間がかかるため、事前に起動しておくこと。

第 4 章

Git によるバージョン管理

オセロゲームの実装を円滑に進めるため、二つのソフトウェア工学的な技術を習得する。ひとつは本章で扱う **Git によるバージョン管理**、もうひとつは次章で扱う **`pytest` による単体テスト**である。これらは、現代のソフトウェア開発では当たり前に使われている技術であり、人工知能のプログラミングを行う上でも避けては通れない。

なお本章は、慶應義塾大学の渡辺宙志先生の GitHub 演習 (渡辺宙志 2020) と、とほほの Git 記事を参考にしている。ここでは本実験で必要な最低限の機能だけを扱うので、より進んだ機能を学びたい人はこれらの資料を参照してほしい。

4.1 バージョン管理とは

プログラムに限らず、実験や研究は、これまでの学習や進捗を踏まえて次のステップに進んでいく営みである。そのためにノートに記録をとり、何を行ったのか、何ができて何ができなかったのかを書き留めることが、効率的にプログラムや実験、研究を進めていく上で重要になる。

プログラミングは、目的の機能を実装するためにコードを書く作業である。すべての機能を一度に実装することはできないので、骨格となるコードを書き上げ、さまざまな機能を一つずつ組み上げていく。しかし、追加した機能が他のコードと相性が悪く全体としてバグが入る、ということは珍しくない。

このような問題に備え、**うまく動いている状態でコードを保存しておき、新しく加えた機能で問題が起きたら元に戻れるようにする**のが、バージョン管理の基本的な考え方である。途中経過を逐次記録することで、問題への対処能力が大きく向上する。

最も素朴なバージョン管理は、ファイルをコピーして名前を変える方法である。しかし、ファイル数が増えると管理が困難になり、どのコピーがどの状態なのか分からなくなる。そこで本実験では、現在広く普及しているバージョン管理システム **Git** を用いる。

> **補足：分散型と集中型**
>
> Git は**分散型**バージョン管理システムであり、サーバが無くてもローカルで履歴を管理できる。これに対し、CVS や Subversion は**集中型**で、サーバ上の履歴に対して全員が読み書きする。分散型は障害に強く、ブランチ操作が軽量という利点がある。

4.2　Git の準備

4.2.1　Git のインストール

Dev Container 内ではすでに Git がインストールされている。バージョンを確認しよう。

```
git --version
```

4.2.2　Git の初期設定

コミットの作者として記録される名前とメールアドレスを設定する。個人の GitHub アカウントがある場合は、それと一致させると後で便利である。

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

設定内容は次のコマンドで確認できる。

```
git config --global --list
```

4.3　Git の基本概念

Git では次の三つの場所を意識する。

1. **ワーキングツリー（working tree）**：ファイルの実体。普段編集している場所。
2. **インデックス（index, staging area）**：次のコミットに含めるファイルの仮置き場。
3. **リポジトリ（repository）**：履歴が保存される場所。`.git/` ディレクトリの中。

ファイルを編集してから履歴に残すまでの流れは次の通りである。

```
編集 → git add（インデックスに登録）→ git commit（履歴に保存）
```

最初は混乱するが、実際に使いながら身につけていけばよい。自転車と同じで、理論を読むよりも繰り返し使ってみることが上達の近道である。

4.4 Git の基本コマンド

以下では、reversi_yourid.py を編集していくシナリオを使って Git の基本を確認する。

4.4.1 git init：リポジトリを作る

新しくバージョン管理を始めるディレクトリで実行する。

```
cd lab_env
git init
```

.git という隠しディレクトリが作られ、ここに履歴が保存される。配布される lab_env.zip にはすでに .git が含まれているので、改めて init する必要は無い。

4.4.2 git status：状態を確認する

```
git status
```

ワーキングツリーとインデックスの状態が表示される。迷ったら、まずこれを実行する習慣をつけるとよい。

4.4.3 git add：インデックスに登録する

ファイルを次のコミットに含めるために登録する。

```
git add reversi_yourid.py
```

複数ファイルをまとめて追加する場合は次のようにする。

```
git add reversi_yourid.py test_reversi.py
```

4.4.4 git commit：履歴に保存する

インデックスの内容を、メッセージ付きで履歴に保存する。

```
git commit -m "implement initial board"
```

メッセージは「**何を**したかが分かる短い英文または日本語」で書く。よくある書き方は次のように動詞から始める形である。

- `implement initial board`
- `add board_movable function`
- `fix off-by-one in board_scan`

4.4.5　`git log`：履歴を見る

```
git log
git log --oneline           # 1 行ずつ表示
git log --oneline --graph   # 分岐をグラフ表示
```

4.4.6　`git diff`：変更を確認する

何を変えたのか、コミット前に確認する。

```
git diff                # ワーキングツリーとインデックスの差分
git diff --cached       # インデックスと最新コミットの差分
git diff HEAD           # ワーキングツリーと最新コミットの差分
```

4.4.7　`git checkout` / `git restore`：変更を取り消す

直前のコミット時点に戻したい場合は、

```
git restore reversi_yourid.py
```

を使う。古い `git checkout -- reversi_yourid.py` でも同じことができる。

> **Warning**
>
> `git restore` は **編集中の変更を捨てる** コマンドである。コミットしていない変更は失われるので注意する。

4.5　練習：実際に commit してみる

ここまで読んだら、実際に手を動かして確認しよう。

4.5.1　練習 G1：初回コミットを作る

`lab_env` ディレクトリで以下を実行し、初期状態をコミットせよ。

```
git status
git add reversi_yourid.py
git status
git commit -m "initial commit"
git log --oneline
```

4.5.2　練習 G2：変更を加えて差分を確認する

reversi_yourid.py の先頭にコメント行を 1 つ追加し、次を順に実行せよ。

```
git diff
git add reversi_yourid.py
git diff                # 何も表示されない
git diff --cached       # 差分が表示される
git commit -m "add header comment"
```

4.5.3　練習 G3：間違いを取り消す

reversi_yourid.py を間違って消してしまった、という想定で次を試せ。

```
echo "broken" > reversi_yourid.py   # 中身を破壊
git status                          # 変更が検出される
git restore reversi_yourid.py       # 直前のコミットに戻す
```

4.6　推奨されるコミットの粒度

本実験では、各章の練習が終わるたびに git commit することを推奨する。これにより、後でうまくいかなくなったときに、最後にうまく動いた状態に戻すことができる。

例えば、本書の第 6 章の練習 13 までを実装し終えたら、

```
git add reversi_yourid.py
git commit -m "implement human reversi game (ex.13)"
```

のようにコミットする。最終レポートでは、git log の内容そのものが**開発履歴**として評価対象になるので、こまめにコミットを残すこと。

4.7 まとめ

- Git は分散型バージョン管理システムであり、本実験では `init` / `add` / `commit` / `status` / `log` / `diff` / `restore` を中心に使う。
- 各章の練習が終わるごとに `commit` する習慣をつける。
- 後の章ではテストと組み合わせて、テストが通るようになったらコミットする、という流れで進めていく。

第 5 章

pytest によるテスト駆動

システム開発では、部品として作る関数が**意図した通りに動いていること**が保障されていないと、組み上げたときに予期しない動作をする。バグが発生してから手で動作確認をするのは手間がかかり、何度も繰り返し確認することはさらに大変である。そこで、関数ごとに**期待する入力と出力の組**を書き、それを自動で検証するテストを用意する。これを**単体テスト（unit test）**と呼ぶ。

ソフトウェア工学では、テストはおおまかに次の四段階に分類される。本実験では、コード作成と直結する**単体テスト**に集中する。

段階	概要
単体テスト	関数やクラスなど小さな単位の動作を確認
結合テスト	複数のモジュールをつなげて動作を確認
システムテスト	完成したシステム全体としての動作を確認
運用テスト	実際の利用環境で問題なく使えるかを確認

5.1 pytest のインストールと実行

本実験の Dev Container には、Python の標準的なテストフレームワークである `pytest` が予めインストールされている。インストールされていない場合は、次のコマンドで導入できる。

```
pip install pytest
```

5.2　はじめてのテスト：assert 文

Python には assert **条件** という構文があり、条件が偽のときに AssertionError を発生させる。これだけでも簡単なテストになる。

```
def square(x):
    return x * x

assert square(2) == 4
assert square(-3) == 9
```

スクリプトとして実行し、何も表示されなければテストは通っている。

5.3　pytest 形式のテスト

pytest では次のルールでテストを書く。

- ファイル名は test_*.py または *_test.py
- テスト関数の名前は test_ で始める
- 関数内で assert を使う

例として、test_square.py を作って次のように書く。

```
# test_square.py
from mymath import square


def test_square_positive():
    assert square(2) == 4


def test_square_negative():
    assert square(-3) == 9


def test_square_zero():
    assert square(0) == 0
```

ターミナルで次のように実行する。

```
python -m pytest
```

すべてのテストが通れば次のような表示になる。

```
=========== 3 passed in 0.01s ===========
```

-v オプションをつけると、テスト関数ごとに結果が表示される。

```
python -m pytest -v
```

5.4　失敗するテストの読み方

square 関数を return x + x と間違えて書いてしまった場合、テストは失敗する。

```
FAILED test_square.py::test_square_positive - assert 4 == 4
```

実際の出力には、どの行で失敗したか、左辺と右辺がどのような値だったかが表示される。この出力を見ながら、コードのどこが間違っているかを推測してデバッグする。

5.5　オセロのテスト例（先取り）

第 6 章で実装する initial_board() などの関数に対しては、次のようにテストを書く。本書の各練習問題には動作確認用の assert 文がついているので、そのまま test_* 関数の中身に貼り付けてテストにできる。

```
# test_reversi.py
from reversi_yourid import (
    BOARD_SENTE, BOARD_GOTE, BOARD_EMPTY, BOARD_SIZE,
    initial_board, change_turn,
    board_movable, board_movable_any, board_move,
    board_number_check,
)



def test_initial_board():
    board = initial_board()
    assert board[3][3] == BOARD_GOTE
    assert board[3][4] == BOARD_SENTE
    assert board[4][3] == BOARD_SENTE
    assert board[4][4] == BOARD_GOTE
    assert board[0][0] == BOARD_EMPTY
```

```
def test_change_turn():
    assert change_turn(BOARD_SENTE) == BOARD_GOTE
    assert change_turn(BOARD_GOTE) == BOARD_SENTE


def test_initial_legal_moves():
    board = initial_board()
    moves = [
        (row, col)
        for row in range(BOARD_SIZE)
        for col in range(BOARD_SIZE)
        if board_movable(board, row, col, BOARD_SENTE)
    ]
    assert len(moves) == 4


def test_move_flips_stone():
    board = initial_board()
    board_move(board, 2, 3, BOARD_SENTE)
    assert board[2][3] == BOARD_SENTE
    assert board[3][3] == BOARD_SENTE
```

5.6　開発の流れ

本実験では、次の流れで開発を進めることを推奨する。

1. 練習問題の関数のシグネチャだけ決めて、本体は pass で書く。
2. その関数の動作確認用 assert を test_reversi.py に追加し、pytest で**失敗する**ことを確認する（**Red**）。
3. 関数本体を実装し、pytest で**通る**ことを確認する（**Green**）。
4. コードを整理する（変数名、ループの書き方など）。テストが引き続き通ることを確認する（**Refactor**）。
5. git add して git commit する。

これを **Red-Green-Refactor サイクル** と呼ぶ。最初は冗長に感じるかもしれないが、慣れると「テストが通った」という確証を持って前に進めるようになる。

5.7　まとめ

- 単体テストはコードの安心材料であり、後から壊してしまったときに気付くための仕組みでもある。
- `pytest` は `test_` で始まる関数の中で `assert` するだけで使える。
- 本書の練習問題には動作確認用の `assert` がついているので、そのままテストに転用できる。
- 各練習が終わるごとに、`pytest` でテストが通ることを確認してから `git commit` する。

第 6 章

ゲームと探索

6.1 ゲームとは

ゲーム（Game）という言葉は、複数のプレイヤーがルールに従って行う競技や遊びを指すことが多いが、一人で行うゲームも含まれる。将棋、チェス、麻雀などのボードゲーム・カードゲームから、スポーツまでが「ゲーム」に含まれる。オリンピックが Olympic Games と呼ばれることからも分かるように、ゲームの範疇は非常に広い。オリンピック競技には個人記録を求める種目もあるが、どこまで頑張るか、頑張らないかは他競技者との駆け引きであり、広く「ゲーム」とみなせる。

このようなゲームの抽象的な要素をモデル化したのが**ゲーム理論（Game Theory）**である。ゲーム理論は、フォン・ノイマンとモルゲンシュテルンによって 1944 年に発表された『ゲームの理論と経済行動（Theory of Games and Economic Behavior)』で初めて体系化された。遊びとしてのゲームというよりは、経済学・社会学などの分野での人間の行動を分析するための理論である。今日ではネットワーク科学とも結びつき、社会現象や経済予測などの分析にも用いられている。

以下では、このゲーム理論に応じた定義に基づき、概要を説明する。

6.2 ゲームの分類

ゲームを整理し解法を導くには、対象を分類する必要がある。分類の代表的なものさしを以下に示す。

- **プレイヤーの人数**：将棋などは二人、麻雀は四人。人数が増えるほど複雑になる。
- **情報の完全性**：将棋などは盤面や持ち駒などの状態が全て把握できる。一方ポーカーでは相手の手札が見えない。情報が欠落するほどアルゴリズムは非決定的になる。
- **状態の有限性**：状態の多様性が有限かどうか。状態数が多いほど処理量も大きい。

- **コスト総和の一定性**：プレイヤーの利害が互いに相反するかどうか。協力で利得が増やせるものを**非零和ゲーム**、誰かの損が必ず別の誰かの得になるものを**零和ゲーム**と呼ぶ。

本実験で扱うオセロゲームは、上記の分類によると**二人完全情報有限零和ゲーム** である。

6.3　ゲームの解析

ゲームの本質を解明するには、次の二点が分かればよい。

- プレイヤーはどう行動すべきか？（**ゲームのアルゴリズム**）
- ゲームの最終結果はどうあるべきか？（**最終目標**）

すなわち、各局面で最もふさわしい一手を選び、それを繰り返して最終的に勝利へと導くことが望まれる。

ここで問題となるのが、最終目標である局面の状態と、中間の局面の状態とのギャップである。ある程度の大きさのゲームでは、状態数は計算機のメモリに乗らないほど膨大になる。たとえばオセロの状態数は 10^{28} という報告があり、終局までの全状態を計算したり保存したりするのは現実的ではない。結局、ある局面で次の手を決めるとき、**最終的にどのような終局になるかが分からない**まま判断しなければならない。

このような状況で人間（および計算機）が取る戦略は、**有限の数手だけ先を読み、その範囲内で最良の手を選ぶ**ことである。最良の手を選ぶには、そのゲームに応じた**評価関数**が必要であり、ヒューリスティックなアルゴリズム（ゲームの世界では**定跡**と呼ばれる）を導入することも有効である。

以下では、二人完全情報有限零和ゲームに対する一般的な手法を説明する。

6.4　状態の表現と探索木

オセロは二人が交互に手を打ち合うゲームである。先手の最初の打ち手を第 1 手とすると、後手が第 2 手、また先手が第 3 手……というふうに、手数の偶奇でどちらが打つかが決まる。

ある手を打った直後の状態を頂点とし、次に打てる手を枝として展開していくと、ゲームの状態遷移を表す**木**が出来上がる（図 Figure 6.1）。

ゲームを計算機で解く場合、このようなゲーム木を作り、その上で最適な解を探索する。厳密にはゲーム状態は**木**ではなく**グラフ**になるが、これは、循環劫や千日手をルールで排除したり、同じ状態を別の頂点で扱うことで木として表現できる。

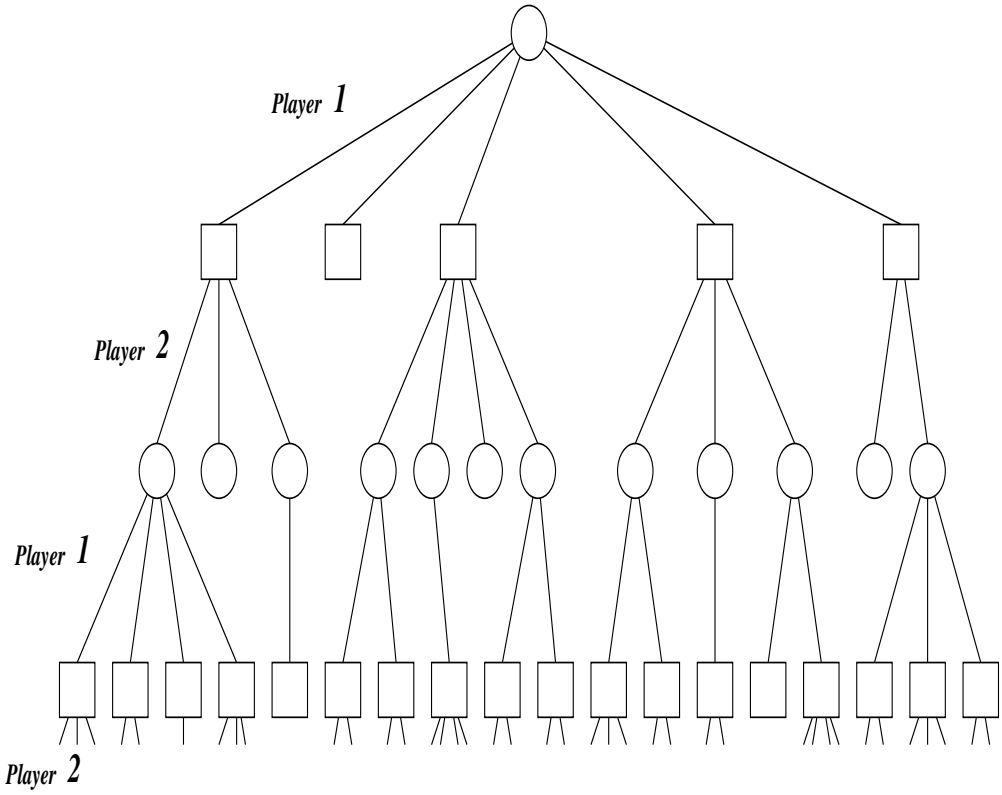

Figure6.1: ゲーム木の一例

ゲームの進行は、ゲーム木の根（root）から葉（leaf）へ向かう状態遷移として表せる。オセロで自分が勝利するには、葉の状態で自分の石数を相手より多くすることが目標になる。すなわち、**最終局面に到達する可能性の高い手を採用する**ことが要求される。

なお、現在の手から葉まで辿って自分が勝つことが確定できることを「**必勝法が存在する**」という。たとえばコネクトフォー（先手必勝）、五目並べ（先手必勝）、6×6 オセロ（後手必勝）などが知られている。

次に、ゲーム木を使って次の一手を決定する一般的手法を説明する。

6.5　minimax 法

この種のゲームは、自分と相手の成績争いである。自分が不利な手を打つと、自動的に相手が有利になる。よって、相手はできるだけ相手に有利な手を打つし、自分はできるだけ自分に有利な手を打つはずである。

これらをまとめると、

- 自分の番では最大利得の手を選ぶ（**max**）

- 相手の番では自分にとって最小利得の手を選ぶ（**min**）

を交互に繰り返すことになる（図 Figure 6.2）。

これが**ミニマックス法（minimax）**であり、先手・後手の双方が互いに最善を尽くすと仮定したうえで、自分にとって最も有利になるように手を選ぶ戦略である。

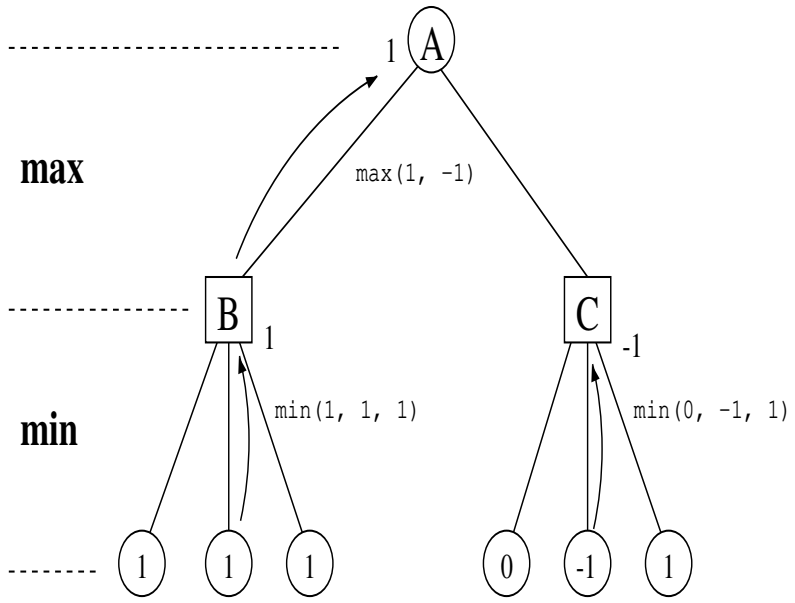

Figure6.2: ミニマックス法

具体的には次のように評価する。ゲーム木の葉では勝敗が決まっているので、

- 先手が勝つ葉に 1
- 後手が勝つ葉に -1
- 引き分けの葉に 0

を与える。それ以外の頂点（葉に至る前）は、

- 先手番なら子の値の**最大**を採用
- 後手番なら子の値の**最小**を採用

と再帰的に計算する。最終的に根（root）に 1 がつけば先手必勝、-1 がつけば後手必勝、0 なら引き分けとなる。

このミニマックス法を計算機で実現するには、木全体を**後順走査（post-order）**しながら各頂点の評価値を決めればよい。後順走査の擬似コードを示す。

```
def post_order(n, level):
    """後順走査の枠組み（擬似コード）"""
```

```
    if n に子供が存在する:
        n の子供を n_1, n_2, ..., n_k とする
        for i in range(1, k + 1):
            post_order(n_i, level + 1)
    else:
        n に対する処理を行う
```

これを minimax に拡張すると次のようになる。

```
def minimax(node, turn, depth):
    """minimax 法（擬似コード）"""
    if node の状態 == 先手が勝ち:
        return 1
    elif node の状態 == 後手が勝ち:
        return -1
    elif node の状態 == 引き分け:
        return 0

    if depth == 0:
        return 局面の評価関数(node)

    現在の局面から 1 手進めた状態を a_1, ..., a_k とする
    if turn == 先手:
        return max(minimax(a_i, 次の手番, depth - 1) for a_i in 子)
    else:
        return min(minimax(a_i, 次の手番, depth - 1) for a_i in 子)
```

このアルゴリズムは有用ではあるが、**探索しなくてもよい頂点まで探索してしまう**という無駄がある。限られた計算機の記憶領域や制限時間のもとでは、この無駄を抑える工夫が望まれる。

6.6　α β 枝刈り（先取り）

ミニマックス法で行われる無駄な探索のうち、「この枝はどうせ親の選択に影響しない」と判明した時点で打ち切るのが**α β 法（alpha-beta pruning）**である。

α β 法は、

- 先手の番でこれまでに保証できる下限値 α
- 後手の番でこれまでに保証できる上限値 β

の二つを管理し、探索中に $\alpha \geq \beta$ になった枝の探索をスキップする（図 Figure 6.3）。

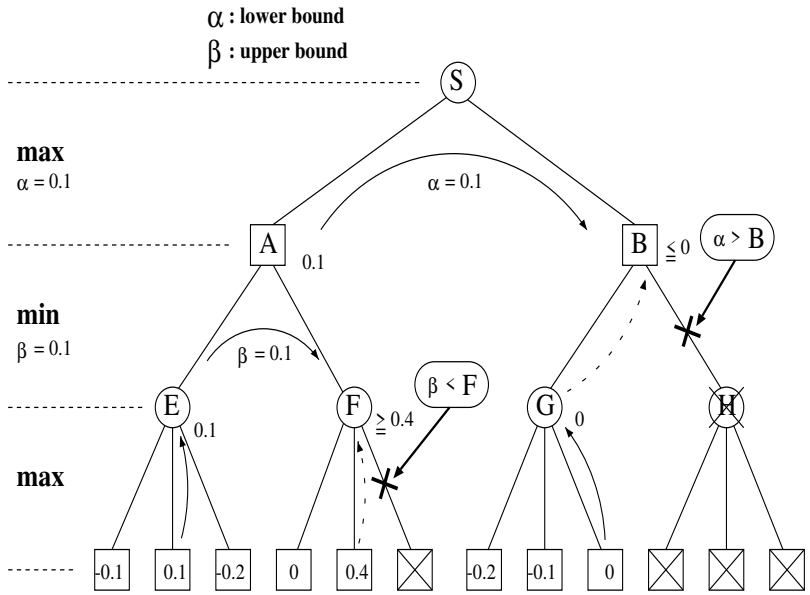

Figure6.3: アルファ・ベータ法の実行例

> **Note**
>
> α β 法は、深く読む場合に効果が大きい。1 手読みでは枝刈りは全く起きず、むしろ余分な比較が増える。理想的な枝刈りでも計算量はおよそ半分にしかならないが、それを使って読みの深さを 2 倍に増やせれば、読みの確かさは飛躍的に向上する。

α β 法の擬似コードを示す。詳細は第 8 章で扱う。

```
def alpha_beta(node, turn, depth, alpha, beta):
    """α β 法（擬似コード）"""
    if node の状態 == 先手が勝ち:
        return 1
    elif node の状態 == 後手が勝ち:
        return -1
    elif node の状態 == 引き分け:
        return 0
    if depth == 0:
        return 局面の評価関数(node)

    現在の局面から 1 手進めた状態を a_1, ..., a_k とする
    for i in range(1, k + 1):
```

```
        if alpha is not None and beta is not None and alpha >= beta:
            break
        v = alpha_beta(a_i, 次の手番, depth - 1, alpha, beta)
        if turn == 先手 and (alpha is None or v > alpha):
            alpha = v
        if turn == 後手 and (beta is None or v < beta):
            beta = v
    return alpha if turn == 先手 else beta
```

なお、ミニマックス法もαβ法も、「相手が間違える可能性まで考えて大胆に攻める」というような戦略はとらない。相手が常に最善を尽くすという仮定のもとで動くため、ある意味**消極的な戦略**であるとも言える。

6.7　評価値（コスト）の計算

ミニマックス法を使ってすべての終局から根まで遡れば、原理的にはゲームに勝てる。しかし、計算機の容量・時間の制約から、ゲーム全体を解析できるのは小さなゲームに限られる。たとえば将棋の場合、ゲームが終了するまでに 100 手以上かかり、各局面で 10 種類以上の手が考えられるので、頂点数は少なくとも 10^{100} になる。これを全て計算するのは到底不可能である。オセロでも状態数は 10^{28} ほどあるので、実際にはある程度の深さまで読んで打ち切る。

すなわち、完全な勝敗が決定しない段階で**何らかの評価値**を付ける必要がある。評価値は、先手有利から後手有利まで $-1 <$ 評価値 < 1 の実数値とすることが多い。この値の付け方には**ゲームに関する知識**が不可欠であり、たとえば将棋なら駒の損得、王将の守備、攻め駒の体勢などを評価する。オセロでは、角の有無、辺の安定性、可動性などを使う。これらを定量化したものが**評価関数（evaluation function）**であり、本実験の主題のひとつである。

また、読みを打ち切る深さも重要な要素である。深く読むほど評価が正確になるのは自明だが、計算時間も指数的に増加する。評価関数の精度と探索深さは**組み合わせて**設計する必要がある。

6.8　まとめ

- 二人完全情報有限零和ゲームは、ゲーム木と minimax 法によって扱える。
- 状態空間が大きすぎる場合は、有限の深さで打ち切り、評価関数で局面を数値化する。
- 探索の無駄は αβ 枝刈りで削れる。

- 評価関数の設計と探索の深さの選択が、強さを決める二大要素である。

第 7 章

オセロゲームの基本実装

本章では、Python でオセロゲームの基本機能を実装する。ここで作成する関数は、後の章でゲーム木探索、minimax 法、α β 法、評価関数の改良にそのまま再利用する。

各課題は、`reversi_yourid.py` に順に追加していく。動作確認用の `assert` 文がついている練習は、そのまま `test_reversi.py` にテストとして追加し、`pytest` で通ることを確認しながら進めること。

7.1 本章の到達目標

- 盤面を表現する。
- 手番を交代する。
- 盤面を表示する。
- 合法手を判定する。
- 石を反転する。
- パス、終局、勝敗を判定する。
- **人間対人間**のオセロゲームを動かす。

7.2 オセロゲームのルール

オセロは、8×8 の盤上で先手と後手が交互に石を打ち、最終的に石の数の多い方が勝ちとなる二人ゲームである。本実験では、先手を `1`、後手を `-1`、空きを `0` として盤面を表現する。

ルールを確認しておく。

1. 最初に盤の中央に先手・後手の石を 2 個ずつ交差するように置く。
2. 相手の石をはさめる場所にだけ自分の石を置ける。

3. はさんだ相手の石はすべて自分の石に反転する。
4. 先手・後手が交互に 1 石ずつ打つ。
5. 打てる場所がない場合はパスとなる。
6. 打てる場所がある場合、自分の意思でパスすることはできない。
7. 盤面がすべて埋まるか、両者とも打てなくなれば終了する。

「**はさむ**」とは、自分の打った石と既に盤面にある自分の石が、縦・横・斜めの 8 方向のいずれかに並び、その間に相手の石が 1 つ以上連続して存在することをいう。

> **Tip**
>
> 旧 LISP 版の教材では、最初に 4×4 の盤で動作確認をしてから 8×8 に拡張する流れを採用していた。本書では `BOARD_SIZE` を定数で変更すれば 4×4 にも切り替えられるようコードを書く。小さな盤でデバッグすると合法手や反転を目視で確認しやすい。

7.3　盤面の表現

盤面は**リストのリスト**で表す。外側のリストが行、内側のリストが列である。Python の添字は `0` から始まるが、画面表示と入力では人間に分かりやすいように `1` から `8` の番号を使う。

```
board[row][col]
```

例えば、`board[3][4]` は 4 行 5 列目を表す。

7.3.1　例題：定数を定義する

盤面の大きさ、石の値、表示文字、8 方向を表す定数を定義する。

```
BOARD_SIZE = 8
BOARD_SENTE = 1
BOARD_GOTE = -1
BOARD_EMPTY = 0

BOARD_SENTE_STR = "*"
BOARD_GOTE_STR = "o"
BOARD_EMPTY_STR = "-"

BOARD_SYMBOL = {
    BOARD_SENTE: BOARD_SENTE_STR,
```

```
    BOARD_GOTE:  BOARD_GOTE_STR,
    BOARD_EMPTY: BOARD_EMPTY_STR,
}

BOARD_DIRECTIONS = [
    (0,  1),   # 右
    (1,  1),   # 右下
    (1,  0),   # 下
    (1, -1),   # 左下
    (0, -1),   # 左
    (-1, -1),  # 左上
    (-1, 0),   # 上
    (-1, 1),   # 右上
]
```

本資料では、先手を `*`、後手を `o` と表示する。`BOARD_SENTE` と `BOARD_GOTE` は、盤面上の石を表す値であると同時に、**手番**を表す値としても使う。

7.3.2　練習 1：初期盤面を作る

初期盤面を返す関数 `initial_board()` を定義せよ。戻り値は 8×8 のリストのリストであり、中央 4 マスだけに石が置かれているものとする。

```
def initial_board():
    ...
```

動作確認:

```
board = initial_board()
assert board[3][3] == BOARD_GOTE
assert board[3][4] == BOARD_SENTE
assert board[4][3] == BOARD_SENTE
assert board[4][4] == BOARD_GOTE
assert board[0][0] == BOARD_EMPTY
```

ヒント

リスト内包表記を使うと、$N \times N$ の空盤を簡潔に作れる。中央 4 マスだけ後で代入すればよい。

```
board = [[BOARD_EMPTY for _ in range(BOARD_SIZE)] for _ in range(BOARD_SIZE)]
```

7.4　手番と表示

7.4.1　練習 2：手番を交代する

手番 turn を引数に取り、次の手番を返す関数 change_turn(turn) を定義せよ。

```
def change_turn(turn):
    ...
```

動作確認:

```
assert change_turn(BOARD_SENTE) == BOARD_GOTE
assert change_turn(BOARD_GOTE) == BOARD_SENTE
```

7.4.2　練習 3：盤面を表示する

盤面 board を受け取り、現在の盤面を表示する関数 print_board(board) を定義せよ。表示は次の形式に合わせること。

```
R/C 1 2 3 4 5 6 7 8
  1 - - - - - - - -
  2 - - - - - - - -
  3 - - - - - - - -
  4 - - - o * - - -
  5 - - - * o - - -
  6 - - - - - - - -
  7 - - - - - - - -
  8 - - - - - - - -
```

合わせて、現在の手番を表示する補助関数 print_current_turn(turn) も定義せよ。

> ヒント
>
> 1 行目は "R/C 1 2 3 4 5 6 7 8" のように固定文字を表示する。各行の先頭の番号は f" {row + 1}" のように row + 1 を表示する。石の表示は BOARD_SYMBOL[stone] を使えば 1 行で書ける。

7.5　盤面の走査

合法手を判定するには、石を置いた場所から 8 方向を調べる必要がある。ある方向について、次の 3 通りを区別する。

1. 自分の石が出る前に**盤面の外**に出た。
2. 自分の石が出る前に**空きマス**に到達した。
3. 相手の石を 1 つ以上はさんだ後、**自分の石**に到達した。

このうち、相手の石を反転できるのは **3 の場合だけ**である。

7.5.1　練習 4：1 方向に反転できる数を数える

board の (row0, col0) に turn の石を置くと仮定し、(row_inc, col_inc) 方向に反転できる相手の石の数を返す関数 board_scan(board, row0, col0, row_inc, col_inc, turn) を定義せよ。

```
def board_scan(board, row0, col0, row_inc, col_inc, turn):
    ...
```

動作確認:

```
board = initial_board()
assert board_scan(board, 2, 3, 1, 0, BOARD_SENTE) == 1
assert board_scan(board, 2, 2, 1, 1, BOARD_SENTE) == 0
assert board_scan(board, 0, 0, 1, 1, BOARD_SENTE) == 0
```

> ヒント
>
> (row0, col0) そのものは、これから石を置く空きマスである。したがって、走査は (row0 + row_inc, col0 + col_inc) から始める。ループの中では「枠外」「空きマス」「自分の石」「相手の石」の 4 ケースを区別する。自分の石に到達したらそれまでに数えた相手石の数を返す。枠外や空きマスに到達したら、はさめなかったので 0 を返す。

7.5.2　練習 5：合法手を判定する

盤面 board の (row, col) に turn の石を置けるかを判定する関数 board_movable(board, row, col, turn) を定義せよ。そのマスが空きであり、8 方向のいずれかで 1 つ以上反転できれば True を返す。

```
def board_movable(board, row, col, turn):
    ...
```

動作確認:

```
board = initial_board()
assert board_movable(board, 2, 3, BOARD_SENTE)
assert not board_movable(board, 2, 3, BOARD_GOTE)
assert not board_movable(board, 3, 3, BOARD_SENTE)
```

7.5.3　練習 6：どこかに打てるかを判定する

手番 turn が盤面上のどこかに石を置けるかどうかを判定する関数 board_movable_any(board, turn) を定義せよ。

```
def board_movable_any(board, turn):
    ...
```

動作確認:

```
board = initial_board()
assert board_movable_any(board, BOARD_SENTE)
assert board_movable_any(board, BOARD_GOTE)
```

7.6　盤面の更新

7.6.1　練習 7：石を置き、反転する

盤面 board の (row, col) に turn の石を置き、8 方向について反転できる石をすべて反転する関数 board_move(board, row, col, turn) を定義せよ。この関数は**盤面を直接更新してよい**。

```
def board_move(board, row, col, turn):
    ...
```

動作確認:

```
board = initial_board()
board_move(board, 2, 3, BOARD_SENTE)
assert board[2][3] == BOARD_SENTE
assert board[3][3] == BOARD_SENTE
```

> **後の探索でのコピー**
>
> 後の章でゲーム木探索を行う際は、**元の盤面を壊さないようにコピーを作ってから** board_move() を呼ぶ。盤面はリストのリストなので、new_board = [row[:] for row in board] のように各行をコピーすれば十分である（深いコピーである copy.deepcopy より速い）。

7.7　終局と評価

7.7.1　練習 8：石の数を数える

盤面 board にある turn の石の数を数える関数 board_number_check(board, turn) を定義せよ。

```
def board_number_check(board, turn):
    ...
```

動作確認:

```
board = initial_board()
assert board_number_check(board, BOARD_SENTE) == 2
assert board_number_check(board, BOARD_GOTE) == 2
```

7.7.2　練習 9：盤面の基本評価値を返す

石数に基づく評価関数 board_eval(board) を定義せよ。評価値は次の式で求める。

$$\frac{\text{先手の石数} - \text{後手の石数}}{\text{先手の石数} + \text{後手の石数}}$$

この値は、先手が有利なら正、後手が有利なら負、互角なら 0 になる。分母が 0 になるとき（理論上は起きない）は 0.0 を返すようにしておく。

```
def board_eval(board):
    ...
```

動作確認:

```
board = initial_board()
assert board_eval(board) == 0.0
```

7.7.3　練習 10：盤面の状態を判定する

盤面 board と手番 turn を受け取り、現在の状態を返す関数 board_state(board, turn) を定義せよ。返す値は次の 3 通りである。

turn が打てるか	相手が打てるか	戻り値	意味
はい	–	None	ゲーム継続中
いいえ	はい	"pass"	パス
いいえ	いいえ	board_eval(board)	終局（評価値）

```
def board_state(board, turn):
    ...
```

> ⚠ 戻り値の型に注意
>
> board_state() は **None**、**str (“pass”)**、**float (評価値)** の 3 種類の型を返す。これを受ける側では、isinstance(state, float) で終局を判定する。評価値が偶然 0.0 であっても isinstance(state, float) は True になるので、if state: のような偽値チェックは使わない。

7.7.4　練習 11：結果を表示する

盤面 board と board_state() の終局評価値 state を受け取り、最終盤面、勝者、石数を表示する関数 game_result(board, state) を定義せよ。

```
def game_result(board, state):
    ...
```

表示例:

```
Game Won by Sente *
Score is 35 - 29
```

state が 0 のときは Draw と表示する。

7.8　人間による着手

7.8.1　練習 12：人間の入力で石を置く

盤面 board、手番 turn、手数 count を受け取り、人間から行と列を入力させて石を置く関数 play_by_human(board, turn, count) を定義せよ。入力は 4 3 のように、行番号と列番号を空白区切りで受け取る。入力された番号は 1 から 8 であり、盤面の添字に使うときは 1 を引く。

```
def play_by_human(board, turn, count):
    ...
```

満たすべき条件は次の通りである。

- 合法手がない場合は何も入力させずに戻る（呼び出し側でパス処理を行う）。
- 入力が 2 つの整数でない場合は再入力させる。
- 盤面外、または合法手でない場合は再入力させる。
- 合法手であれば board_move() で盤面を更新して戻る。

ヒント

入力文字列を整数 2 つに変換するには次のようにする。

```
inp = input(f"({count}) Enter row and col for {BOARD_SYMBOL[turn]}: ")
parts = inp.strip().split()
if len(parts) != 2:
    continue
try:
    row = int(parts[0]) - 1
    col = int(parts[1]) - 1
except ValueError:
    continue
```

7.9　人間対人間のゲーム

7.9.1　練習 13：汎用ゲーム関数を定義する

第 7 章でコンピュータ対コンピュータの実装を行うことを見据えて、ここでは**汎用のゲーム関数**を一つ定義する。othello(sente_gote, yomi_depth) は次の通りに振る舞う。

sente_gote	意味
BOARD_SENTE	コンピュータが先手、人間が後手
BOARD_GOTE	人間が先手、コンピュータが後手
BOARD_EMPTY	先手も後手もコンピュータ
その他（例：None）	先手も後手も人間（人間対人間）

yomi_depth はコンピュータの先読み深さである。人間対人間のときは使われない（None でよい）。

```
def othello(sente_gote, yomi_depth=None):
    board = initial_board()
    turn = BOARD_SENTE    # 初期手番は常に先手
    count = 1
    ...
```

処理の流れは次の通りである。

1. state = board_state(board, turn) で現在の状態を確認する。
2. state が float なら終局なので game_result(board, state) を呼んで終了する。
3. state == "pass" なら、パスのメッセージを表示し、手数を増やさずに手番だけ交代する。
4. それ以外なら、盤面を表示し、コンピュータ担当なら play_by_machine(board, turn, count, yomi_depth)、人間担当なら play_by_human(board, turn, count) を呼ぶ。
5. 手番を交代し、手数を進める。

play_by_machine は次章で実装するので、ここでは未定義のままにし、sente_gote が「コンピュータ担当を含まない値」（例えば None）のときだけ動かせるようにしておく。

> ❗ 重要：初期手番は常に先手
>
> turn の初期値は BOARD_SENTE である。sente_gote の値ではない点に注意せよ（sente_gote はコンピュータの担当を表すだけ）。

7.9.2　練習 14：人間対人間で動作確認する

othello(None) を実行し、人間対人間で初期盤面から最後まで実行できることを確認せよ。

ここまで実装できたら、pytest でテストを通し、Git でコミットしておくこと。

```
python -m pytest -v
git add reversi_yourid.py test_reversi.py
git commit -m "implement human reversi game (ex.1-14)"
```

7.10　まとめ

- 本章では、盤面表現・走査・合法手判定・反転・終局判定・人間入力という、オセロゲームの**ルール部分**を実装した。
- 状態判定 `board_state()` は **`None`** / **`"pass"`** / **`float`** の 3 通りの値を返す点に注意する。
- 汎用ゲーム関数 `othello(sente_gote, yomi_depth)` の枠組みを用意したので、次章でコンピュータの着手 `play_by_machine` を実装すれば、人間対コンピュータ・コンピュータ対コンピュータが自動的に動くようになる。

第 8 章

minimax 法によるゲーム木探索

本章では、人間が入力する代わりに、コンピュータが次の手を選ぶ仕組みを実装する。現在の盤面から合法手をすべて展開すると、1 手先の盤面の集合が得られる。さらに、その各盤面から相手の合法手を展開していくと、ゲーム木が得られる。

本章の到達目標は、minimax 法を用いてコンピュータが手を選べるようにすることである。

8.1 節点のデータ構造

ゲーム木の節点には、盤面、その盤面に至るために打った手、評価値などを保存する。Python では**クラス**を使って表現する。

8.1.1 例題：Node クラスを定義する

```
class Node:
    def __init__(self, board, turn, row, col, value):
        self.board = board
        self.turn = turn
        self.row = row
        self.col = col
        self.value = value
```

各属性の意味は次の通りである。

- board: その節点に対応する盤面（リストのリスト）。
- turn: その盤面に至る手を打った**手番**（BOARD_SENTE または BOARD_GOTE）。
- row, col: その盤面に至るために置いた位置（0 始まり）。

- value: その節点に与えられた評価値。

> **Note**
>
> ここでの turn は「この節点の盤面で次に打つ手番」ではなく、「**この盤面に至るために直前に打った手番**」である点に注意する。初期局面ではまだ着手がないので、turn には便宜上 BOARD_SENTE、row/col には 0 などの仮の値を入れておく。
>
> ```
> root = Node(initial_board(), BOARD_SENTE, 0, 0, 0.0)
> ```

8.2　節点の操作関数

探索に必要な操作は大きく 3 つである。

- 節点が**終局**、**パス**、**継続中**のどれであるかを判定する。
- 節点を**評価**する。
- 節点から**子節点**を生成する。

8.2.1　例題：節点の状態を返す

節点 node の盤面に対して、手番 turn の状態を返す関数 node_state(node, turn, count) を定義する。count は後で局面の段階に応じて評価関数を切り替えるために受け取る引数で、現時点では使わない。

```
def node_state(node, turn, count):
    return board_state(node.board, turn)
```

8.2.2　例題：節点を評価する

まずは単純化のため、探索を打ち切った節点の評価値として石数評価 board_eval() を使う。

```
def eval_node(node, turn, count):
    return board_eval(node.board)
```

第 9 章で、この関数をより強い評価関数に差し替える。

8.2.3　練習 15：子節点を生成する

手番 turn において、節点 parent から打てるすべての手を調べ、その手を打った後の子節点のリストを返す関数 expand_node(parent, turn) を定義せよ。

```
def expand_node(parent, turn):
    children = []
    ...
    return children
```

実装上の注意:

- parent.board を**直接変更してはいけない**。
- 合法手ごとに new_board = [row[:] for row in parent.board] で盤面をコピーする。
- コピーした盤面に board_move(new_board, row, col, turn) を適用する。
- Node(new_board, turn, row, col, 0.0) を作って children に追加する。

動作確認:

```
root = Node(initial_board(), BOARD_SENTE, 0, 0, 0.0)
children = expand_node(root, BOARD_SENTE)
assert len(children) == 4
assert all(isinstance(child, Node) for child in children)
```

8.3 minimax 法の実装

minimax 法では、**先手は評価値を最大化**し、**後手は評価値を最小化**する。評価値は「先手が有利な盤面ほど大きく、後手が有利な盤面ほど小さい」ように設計しているからである。

探索の基本方針は次の通りである。

- 終局なら、その盤面の評価値を返す。
- 読みの深さが 0 なら、評価関数で評価した値を返す。
- パスなら、手番だけを交代して同じ深さで探索を続ける。
- 先手番なら、子節点の中で評価値が**最大**のものを選ぶ。
- 後手番なら、子節点の中で評価値が**最小**のものを選ぶ。

8.3.1 練習 16：minimax 本体を定義する

節点 node、手番 turn、手数 count、読みの深さ depth を受け取り、最善手に対応する節点を返す関数 minimax(node, turn, count, depth) を定義せよ。

```
def minimax(node, turn, count, depth):
    state = node_state(node, turn, count)
    ...
```

以下の場合分けを行う。

- state が float なら終局である。node.value = state を入れて node を返す。
- depth == 0 なら、node.value = eval_node(node, turn, count) を入れて node を返す。
- state == "pass" なら、手番を交代して同じ節点から探索を続ける（depth は変更しない）。
- それ以外なら、expand_node() で子節点を作り、minimax_children() に渡す。

float かどうかは次のように判定する。

```
if isinstance(state, float):
    ...
```

8.3.2　練習 17：子節点の中から最善手を選ぶ

children の各節点について再帰的に minimax() を呼び出し、手番に応じて最大または最小の値を持つ節点を返す関数 minimax_children(children, turn, count, depth) を定義せよ。

```
def minimax_children(children, turn, count, depth):
    new_turn = change_turn(turn)
    new_depth = depth - 1
    new_count = count + 1
    val = None
    result_node = None
    for child in children:
        v = minimax(child, new_turn, new_count, new_depth).value
        if val is None or (turn == BOARD_SENTE and v > val) or (turn == BOARD_GOT
            val = v
            result_node = Node(child.board, child.turn, child.row, child.col, v)
    return result_node
```

戻り値は、評価値そのものではなく、**次に打つ手に対応する Node** とする。返す節点の value には、探索で得られた評価値を保存する。

動作確認:

```
root = Node(initial_board(), BOARD_SENTE, 0, 0, 0.0)
best = minimax(root, BOARD_SENTE, 1, 2)
assert isinstance(best, Node)
assert board_movable(initial_board(), best.row, best.col, BOARD_SENTE)
```

8.4　コンピュータの着手

8.4.1　練習 18：コンピュータが石を置く

盤面 board、手番 turn、手数 count、読みの深さ depth を受け取り、minimax 法で選んだ手を盤面に反映する関数 play_by_machine(board, turn, count, depth) を定義せよ。

```
def play_by_machine(board, turn, count, depth):
    root = Node(board, turn, 0, 0, 0.0)
    best_node = minimax(root, turn, count, depth)
    row, col = best_node.row, best_node.col
    board_move(board, row, col, turn)
    print(f"({count}) My move is {row + 1} {col + 1}.")
```

best_node.row, best_node.col が次に打つ位置である。1 始まりに直して表示する点に注意する。

8.5　人間対コンピュータ、コンピュータ対コンピュータ

8.5.1　練習 19：8×8 盤で動作を確認する

第 6 章の練習 13 で定義した othello(sente_gote, yomi_depth) は、play_by_machine を呼び出すように作っておけば、本章の play_by_machine がそのまま使われる。8×8 盤で次のすべてが動くことを確認せよ。

```
othello(BOARD_GOTE, 3)    # 人間先手 vs コンピュータ後手
othello(BOARD_SENTE, 3)   # コンピュータ先手 vs 人間後手
othello(BOARD_EMPTY, 2)   # コンピュータ vs コンピュータ
```

確認すべき項目は次の通り。

- 初期盤面で先手の合法手が 4 つである。
- 人間対人間で最後まで実行できる（前章で確認済み）。
- 人間対コンピュータで最後まで実行できる。
- コンピュータ対コンピュータで最後まで実行できる。
- パスが発生しても手数の扱いが破綻しない。
- 探索深さを大きくすると思考時間が伸びる（深さ 3 と 5 を比較）。

確認できたら、Git でコミットしておくこと。

```
git add reversi_yourid.py test_reversi.py
git commit -m "implement minimax reversi player (ex.15-19)"
```

8.6 まとめ

- ゲーム木の節点を Node クラスで表現し、expand_node() で子節点を作り、minimax() で再帰的に最善手を探索した。
- パスは「手番だけ交代して同じ深さで再帰呼び出し」で扱う。
- 終局は state が float であることで判定する。
- 探索深さを大きくすると、計算時間は指数的に増える。次章ではこれを抑える α β 枝刈りを導入する。

第 9 章

α β 枝刈り

minimax 法は、すべての子節点を探索してから最善手を決める。しかし、探索の途中で「この枝は最終的に選ばれない」と分かる場合がある。そのような枝の探索を省略する方法が**α β 枝刈り（alpha-beta pruning）**である。

α β 法は、**同じ評価関数・同じ子節点の順序**であれば minimax 法と同じ手を返す。しかし、探索する節点数を減らせるため、同じ計算予算でより深い読みが可能になる。

9.1 αとβ

α β 法では、探索中に次の 2 つの値を持ち回る。

- `alpha`：先手がこれまでに**保証できている下限値**
- `beta`：後手がこれまでに**保証できている上限値**

探索中に `alpha >= beta` になったら、それ以上その枝を調べても親の選択は変わらない。そのため、そこで探索を打ち切る。

9.2 α β 法の本体

9.2.1 練習 20：`alpha_beta()` を定義する

節点 `node`、手番 `turn`、手数 `count`、読みの深さ `depth`、下限 `alpha`、上限 `beta` を受け取り、最善手に対応する節点を返す関数 `alpha_beta(node, turn, count, depth, alpha, beta)` を定義せよ。

```
def alpha_beta(node, turn, count, depth, alpha, beta):
    state = node_state(node, turn, count)
    ...
```

基本構造は minimax() と同じである。

- 終局なら、node.value = state を設定して node を返す。
- 深さ限界なら、node.value = eval_node(node, turn, count) を設定して node を返す。
- パスなら、手番を交代して同じ深さで再帰呼び出しする。
- それ以外なら、子節点を生成して alpha_beta_children() で選ぶ。

alpha と beta は、最初はまだ値がないため None を渡す（呼び出し側で alpha_beta(node, turn, count, depth, None, None) のようにする）。

9.3　子節点の探索

9.3.1　練習 21：alpha_beta_children() を定義する

children を順に探索し、α β 枝刈りを行いながら最善の節点を返す関数 alpha_beta_children(children, turn, count, depth, alpha, beta) を定義せよ。

```
def alpha_beta_children(children, turn, count, depth, alpha, beta):
    new_turn = change_turn(turn)
    new_depth = depth - 1
    new_count = count + 1
    val = None
    result_node = None
    for child in children:
        v = alpha_beta(child, new_turn, new_count, new_depth, alpha, beta).value
        if turn == BOARD_SENTE:
            if val is None or v > val:
                val = v
                result_node = Node(child.board, child.turn, child.row, child.col,
            if alpha is None or v > alpha:
                alpha = v
        elif turn == BOARD_GOTE:
            if val is None or v < val:
                val = v
                result_node = Node(child.board, child.turn, child.row, child.col,
            if beta is None or v < beta:
                beta = v
        if alpha is not None and beta is not None and alpha >= beta:
```

```
            break
    return result_node
```

各部分の意味を確認する。

- 先手番では、子節点の最大評価値を選び、alpha（下限）を更新する。
- 後手番では、子節点の最小評価値を選び、beta（上限）を更新する。
- 最後に **alpha >= beta** になっていたらループを抜けて以降の子節点を探索しない（**枝刈り**）。

> 注意：None の扱い
>
> 最初は alpha も beta も None である。比較の前に必ず is not None を確認する。これは「まだ何も値がない」状態を表すためのものである。最初の子の評価値が決まれば、alpha（または beta）はその値で初期化される。

9.4　コンピュータの着手に組み込む

9.4.1　練習 22：play_by_machine() をαβ法に切り替える

play_by_machine() で呼び出す探索関数を minimax() から alpha_beta() に変更する。

```
def play_by_machine(board, turn, count, depth):
    node = Node(board, turn, 0, 0, 0.0)
    # best_node = minimax(node, turn, count, depth)
    best_node = alpha_beta(node, turn, count, depth, None, None)
    row, col = best_node.row, best_node.col
    board_move(board, row, col, turn)
    print(f"({count}) My move is {row + 1} {col + 1}.")
```

minimax 法と αβ法は、同じ評価関数と同じ子節点の順序であれば、同じ手を選ぶ。結果が大きく異なる場合は、value の更新、手番の交代、パス処理を確認すること。

> 切り替えを実験で残すコツ
>
> 両方の関数を残して切り替え可能にしておくと、後で比較しやすい。たとえばモジュール変数 USE_ALPHA_BETA = True を導入して、

```
def play_by_machine(board, turn, count, depth):
    node = Node(board, turn, 0, 0, 0.0)
    if USE_ALPHA_BETA:
        best_node = alpha_beta(node, turn, count, depth, None, None)
    else:
        best_node = minimax(node, turn, count, depth)
    ...
```

のように書くと、評価関数を変えずに探索手法だけ切り替えて比較できる。

9.5　探索効率の比較

9.5.1　練習 23：呼び出し回数を数える

minimax 法と α β 法の効率の違いを定量的に確認するため、探索関数が呼ばれた回数を数える。Python では、モジュール内のグローバル変数を使うと簡潔に書ける。

```
NODE_CHECK_COUNT = 0

def node_check_begin():
    global NODE_CHECK_COUNT
    NODE_CHECK_COUNT = 0

def node_check():
    global NODE_CHECK_COUNT
    NODE_CHECK_COUNT += 1

def node_check_end():
    print(f"{NODE_CHECK_COUNT} nodes checked.")
```

`minimax()` と `alpha_beta()` の先頭に `node_check()` を呼び、`play_by_machine()` の前後で `node_check_begin()`、`node_check_end()` を呼ぶ。

```
def play_by_machine(board, turn, count, depth):
    node_check_begin()
    node = Node(board, turn, 0, 0, 0.0)
    best_node = alpha_beta(node, turn, count, depth, None, None)
    node_check_end()
    ...
```

9.5.2　練習 24：実験の記録

比較する条件を揃えて、minimax 法と α β 法の探索節点数・実行時間を表に記録せよ。

探索深さ	手法	探索節点数	実行時間 [s]	選んだ手
3	minimax			
3	α β			
4	minimax			
4	α β			
5	minimax			
5	α β			

実行時間は `time.perf_counter()` を使う。

```
import time

start = time.perf_counter()
best_node = alpha_beta(node, turn, count, depth, None, None)
elapsed = time.perf_counter() - start
print(f"{elapsed:.3f} sec")
```

9.5.3　練習 25：考察

次の点について考察せよ。

- 浅い深さでは α β の効果が小さい理由。
- 深い深さでは α β の効果が大きくなる理由。
- 子節点の探索順序を変えると効果はどう変わるか。

確認できたら、Git でコミットしておくこと。

```
git add reversi_yourid.py test_reversi.py
git commit -m "implement alpha-beta pruning (ex.20-25)"
```

9.6　まとめ

- α β 法は、minimax 法と同じ最適解を返しつつ、探索する節点数を減らす。
- 効果が出るのは「ある程度深く読む」場合。理想的な枝刈りでも計算量は約 1/2 にしかならないが、その分深く読むことで強さは大きく向上する。

- 子節点の順序が良ければ枝刈りの効果はさらに大きくなる。これを生かす工夫は応用課題として考えられる。

第 10 章

ヒューリスティック評価関数

minimax 法・α β 法では、終局まで探索できれば勝敗を正確に求められる。しかし、8×8 のオセロを終局まで全探索することは現実的ではない。そのため、ある深さで探索を打ち切り、その時点の盤面に評価値を与える必要がある。

完全な解ではないが、問題に関する**知識**を使って良さそうな値を与える方法を**ヒューリスティック評価**と呼ぶ。本章では、石数だけでない評価関数を段階的に実装する。

10.1 局面の段階

オセロでは、序盤・中盤・終盤で良い手の性質が変わる。旧 LISP 版テキストでも、おおよそ次のように分割して考えてきた。

- **序盤**（1〜20 手）：石を取りすぎず、相手の自由度を下げる。中央の 32 マスでの陣取り。
- **中盤**（21〜40 手）：辺の攻防、中割り・中抜き、可動性。
- **終盤**（41 手〜）：隅の攻防、確定石、できるだけ深く読む。深読みした時点で多くの石を取る。

本実験の完成版では、手数 `count` に応じて評価の重みを変える。

```
def eval_node(node, turn, count):
    board = node.board
    if count < 25:
        ...   # 序盤
    elif count < 40:
        ...   # 中盤
    elif count < 60:
        ...   # 終盤前
```

```
    else:
        return board_eval(board)   # 終盤
```

以下の練習で board_eval2 〜 board_eval5 を順に実装し、最後にこれらを組み合わせて eval_node を完成させる。

10.2　位置による評価（board_eval2）

オセロでは、**角は非常に強く**、**角の隣は危険**になることが多い。そこで、盤面上の各マスに点数を与え、自分の石がある場所の点数を加算し、相手の石がある場所の点数を減算する。

10.2.1　例題：盤面の位置点を定義する

```
BOARD_POINT = [
    [ 30, -12,  0.1, -0.9, -0.9,  0.1, -12,  30],
    [-12, -15, -3,   -3,   -3,   -3,   -15, -12],
    [ 0.1, -3,  0,   -1,   -1,    0,   -3,   0.1],
    [-0.9, -3, -1,   -1,   -1,   -1,   -3,  -0.9],
    [-0.9, -3, -1,   -1,   -1,   -1,   -3,  -0.9],
    [ 0.1, -3,  0,   -1,   -1,    0,   -3,   0.1],
    [-12, -15, -3,   -3,   -3,   -3,   -15, -12],
    [ 30, -12,  0.1, -0.9, -0.9,  0.1, -12,  30],
]
```

この値は**一例**である。角（4 隅）に高い点数、その隣（C 打ち・X 打ちの位置）に大きな負の点数を与えている。最終課題では、各自で値を変更して強さの変化を調べてよい。

10.2.2　練習 26：位置点による評価を実装する

盤面 board と重み omomi を受け取り、位置点に基づく評価値を返す関数 board_eval2(board, omomi) を定義せよ。

```
def board_eval2(board, omomi):
    count = 0.0
    ...
    return count * omomi
```

評価の考え方:

- 先手の石（BOARD_SENTE）なら + BOARD_POINT[row][col] を加える。
- 後手の石（BOARD_GOTE）なら- BOARD_POINT[row][col] を加える（BOARD_GOTE = -1 を掛けることで実現）。
- 空きマスは評価に影響させない。

つまり、各マスについて BOARD_POINT[row][col] * board[row][col] を合計し、最後に omomi を掛ければよい。

動作確認:

```
board = initial_board()
v = board_eval2(board, 3)
assert isinstance(v, float)
# 初期盤面では先手後手の位置点は対称なので 0.0 になるはず
assert v == 0.0
```

10.3　可動性の評価（board_eval3）

現在打てる手が多いほど、次に選べる選択肢が多い。逆に、相手の合法手を減らせる手は有利になりやすい。

10.3.1　練習 27：合法手の位置を評価する

盤面 board、手番 turn、重み omomi を受け取り、turn が打てる場所を評価する関数 board_eval3(board, turn, omomi) を定義せよ。

```
def board_eval3(board, turn, omomi):
    count = 0.0
    for row in range(BOARD_SIZE):
        for col in range(BOARD_SIZE):
            if board_movable(board, row, col, turn):
                count += turn * (BOARD_POINT[row][col] + omomi)
    return count
```

turn が先手なら正方向、後手なら負方向に評価される。omomi は「とにかく合法手があるだけで嬉しい」という基底点であり、これに位置点を加えて重み付けしている。

> 設計上のポイント
>
> board_eval3 は turn の合法手だけを評価する。本当は「自分の合法手数 − 相手の合法手数」を考慮するほうが望ましい。発展課題として、board_eval3 を turn

> と change_turn(turn) の両方について計算し、差を取るように改造してみるとよい。

10.4　確定石の評価（board_eval4）

角を取った石、および**角から連続して同じ色でつながっている石**は、相手に反転されにくい。これを**確定石**と呼ぶ。ここでは簡易的に、4 つの角から縦横方向に連続する同色の石を数える。

10.4.1　練習 28：1 方向の確定石を数える

盤面 board の（row0, col0）から（row_inc, col_inc）方向へ、手番 turn の石が連続している数を数える関数 kakutei_scan(board, row0, col0, row_inc, col_inc, turn）を定義せよ。ただし、戻り値は連続数そのものではなく、**「連続数 × turn の値」**とする。

```
def kakutei_scan(board, row0, col0, row_inc, col_inc, turn):
    row = row0
    col = col0
    length = 0
    while 0 <= row < BOARD_SIZE and 0 <= col < BOARD_SIZE and board[row][col] ==
        length += turn   # 先手なら +1、後手なら -1
        row += row_inc
        col += col_inc
    return length
```

このようにすると、後で board_eval4 の中で「先手の確定石は正、後手の確定石は負」になるよう自然に積み上げられる。

動作確認:

```
board = initial_board()
# 初期盤面では角は空なので、(0, 0) からの確定石は 0
assert kakutei_scan(board, 0, 0, 1, 0, BOARD_SENTE) == 0

# (3, 4) は先手の石。そこから上方向 (-1, 0) には先手 1 個
assert kakutei_scan(board, 3, 4, -1, 0, BOARD_SENTE) == 0   # 上に空
# 自分自身を含めて数えるので、先手 1 個分の +1
# ※ 細かい仕様（自身を含めるかどうか）は実装と合わせて確認すること
```

> **Warning**
>
> kakutei_scan は（row0, col0）自身も turn の石である場合だけ意味のある値を返す。角に石が無い場合は呼び出さないようにする（次の board_eval4 で確認している）。

10.4.2　練習 29：角からの確定石を評価する

4 つの角について、角に石がある場合だけ、縦横の連続石を評価する関数 board_eval4(board, omomi) を定義せよ。

```
def board_eval4(board, omomi):
    count = 0.0
    if board[0][0] != BOARD_EMPTY:
        turn = board[0][0]
        count += kakutei_scan(board, 0, 0, 1, 0, turn)   # 下方向
        count += kakutei_scan(board, 0, 0, 0, 1, turn)   # 右方向
    if board[7][7] != BOARD_EMPTY:
        turn = board[7][7]
        count += kakutei_scan(board, 7, 7, -1, 0, turn)  # 上方向
        count += kakutei_scan(board, 7, 7, 0, -1, turn)  # 左方向
    if board[0][7] != BOARD_EMPTY:
        turn = board[0][7]
        count += kakutei_scan(board, 0, 7, 1, 0, turn)   # 下方向
        count += kakutei_scan(board, 0, 7, 0, -1, turn)  # 左方向
    if board[7][0] != BOARD_EMPTY:
        turn = board[7][0]
        count += kakutei_scan(board, 7, 0, -1, 0, turn)  # 上方向
        count += kakutei_scan(board, 7, 0, 0, 1, turn)   # 右方向
    return count * omomi
```

> **Note**
>
> 角の座標は $(0,0)$, $(0,7)$, $(7,0)$, $(7,7)$ の 4 つ。各角から内側に向かう 2 方向（縦・横）について連続石を数えることで、辺に沿った確定石を素朴に評価している。本来は斜め方向の伝播なども考慮すべきだが、ここでは簡易版にとどめる。

10.5　X 打ち・C 打ちの評価（board_eval5）

角の**斜め隣**のマス（X 打ち）や、角の**辺隣**のマス（C 打ち）は、角が空いているときに打つと相手に角を取られる原因になりやすい。

10.5.1　練習 30：X 打ちを減点する

盤面 board、手番 turn、重み omomi を受け取り、X 打ちを減点する関数 board_eval5(board, turn, omomi) を定義せよ。

```
def board_eval5(board, turn, omomi):
    count = 0.0
    if board[1][1] == turn and board[0][0] == BOARD_EMPTY:
        count += turn * -1
    if board[1][6] == turn and board[0][7] == BOARD_EMPTY:
        count += turn * -1
    if board[6][1] == turn and board[7][0] == BOARD_EMPTY:
        count += turn * -1
    if board[6][6] == turn and board[7][7] == BOARD_EMPTY:
        count += turn * -1
    return count * omomi
```

対象となる X の座標と角の組は次の通り。

X の位置	対応する角
$(1,1)$	$(0,0)$
$(1,6)$	$(0,7)$
$(6,1)$	$(7,0)$
$(6,6)$	$(7,7)$

X の位置に turn の石があり、対応する角が空きなら、turn * -1 を加算する。これにより、turn = BOARD_SENTE のときは負、turn = BOARD_GOTE のときは正の値が積まれる（あとで omomi（正値）を掛けるので、最終的に「先手は減点・後手は加点」となる）。

発展課題

C 打ち（角の辺隣 $(0,1),(1,0),(0,6),(1,7),\ldots$）も同様に減点するよう拡張できる。最終レポートで、「C 打ちを減点項目に追加するとどう変わったか」を実験すると説

> 得力のある考察になる。

10.6　評価関数を組み合わせる

10.6.1　練習 31：eval_node() を改良する

これまで作成した評価関数を組み合わせて、eval_node(node, turn, count) を改良せよ。完成版では次の方針を用いる。

```
def eval_node(node, turn, count):
    board = node.board
    if count < 25:                  # 序盤
        return (board_eval2(board, 3)
                + board_eval3(board, turn, 20)
                + board_eval4(board, 100)
                + board_eval5(board, turn, 500))
    elif count < 40:                # 中盤
        return (board_eval2(board, 3)
                + board_eval3(board, turn, 40)
                + board_eval4(board, 100))
    elif count < 60:                # 終盤前
        return (board_eval2(board, 3)
                + board_eval3(board, turn, 20)
                + board_eval4(board, 100))
    else:                           # 終盤
        return board_eval(board)
```

これは**一つの基準実装**である。より強いプログラムにするには、重み・局面の分割・評価項目を変更し、実験で比較する必要がある。

10.6.2　練習 32：評価関数の効果を確認する

評価関数を変更する前後で、同じ初期盤面・同じ探索深さを使ってコンピュータ同士を対戦させ、結果を比較せよ。

評価関数	探索深さ	先手石数	後手石数	勝者	コメント
board_eval のみ	3				
board_eval2 を追加	3				
board_eval2 + board_eval4	3				

評価関数	探索深さ	先手石数	後手石数	勝者	コメント
完成版	3				

確認できたら、Git でコミットしておくこと。

```
git add reversi_yourid.py test_reversi.py
git commit -m "improve board evaluation (ex.26-32)"
```

10.7　次章以降への接続

本章で扱った評価関数の重み（omomi）は、すべて**人間の直感**で決めた値である。これを次のように**自動化**することができる。

- 第 10 章**手動チューニング**：実験を繰り返して人間が改良する（次章）。
- 第 11 章**メタヒューリスティクス**：山登り法・焼きなまし法・遺伝的アルゴリズムなどの探索アルゴリズムで重みを自動最適化する。
- 第 12 章**機械学習**：自己対戦データから評価関数そのものを学習する。

本実験では、まず本章の手動の評価関数で動くプログラムを完成させ、その後の章でこれらの自動化を体験する。それぞれの章は AI の歴史的発展（推論 → 最適化 →学習）に対応しており、同じ問題を異なる視点で解くことの面白さを味わえるはずである。

10.8　まとめ

- 評価関数はオセロの強さを決める中核である。
- 位置点・可動性・確定石・X 打ちなど、ゲームの知識を分けて関数化し、重み付き和で組み合わせるのが基本戦略。
- 序盤・中盤・終盤で重みを変えることで、局面に応じた指針に切り替えられる。
- 改良の効果は**実験で確かめて**、結果として記録する習慣をつける。
- 手動チューニングで強い AI を作るのは限界がある。次章以降では、自動最適化や機械学習でこの限界を超える方法を学ぶ。

第 11 章

評価関数の改良と最終課題

本章では、これまでに完成したオセロプログラムをより強くするため、評価関数を改良する。最終課題では、単にコードを完成させるだけでなく、**どのような考えで評価関数を設計し、その結果がどう変わったか**を実験データに基づいて説明することが求められる。

11.1　改良の方針

評価関数の改良例を示す。

- 角を取る評価をさらに大きくする。
- 角が空いているときの X 打ち・C 打ちを強く減点する。
- 自分の合法手数を増やし、相手の合法手数を減らす（可動性）。
- 終盤では石数評価の重みを大きくする。
- 空きマスが少なくなったら探索深さを動的に深くする。
- 辺の安定性を評価する（辺の中央の C 打ち、辺の端の固定石など）。
- 手番ごとに評価の重みを変える（自分の手番か相手の手番かでバイアスを変える）。
- 旧 LISP 版テキストにあった「中割り」「手止まり」のような盤面パターンを評価項目に組み込む。

これらを**一度にすべて入れると、何が効いたのか分かりにくい**。まず 1 つの改良だけを入れて比較し、効果がありそうなら次の改良を追加する、という増分的な進め方を推奨する。

11.2　実験の進め方

評価関数を変更するときは、次の手順を守る。

1. 変更前のコードで対戦結果を記録する。
2. Git で現在の状態をコミットする。

3. 評価関数を **1 か所**変更する。
4. `pytest` でテストを実行し、ルール部分が壊れていないことを確認する。
5. 同じ条件で対戦させ、結果を記録する。
6. 良くなったか、悪くなったかを考察する。
7. 良ければコミットする。悪ければブランチを戻すか、結果を記録した上で別の改良を試す。

最低限、次のような表を作って実験記録を残す。

実験	変更内容	探索深さ	先手石数	後手石数	勝者	コメント
1	基準実装	3				
2	角の重みを変更	3				
3	X 打ち減点を変更	3				
4	終盤判定の境界を変更	3				

11.3　ルール部分のテスト

評価関数を改良しても、オセロのルール部分を壊してはいけない。少なくとも次の確認は毎回行う（既に `test_reversi.py` に書いてあれば再利用できる）。

```
def test_initial_board():
    board = initial_board()
    assert board_number_check(board, BOARD_SENTE) == 2
    assert board_number_check(board, BOARD_GOTE) == 2

def test_initial_legal_moves():
    board = initial_board()
    moves = [
        (row, col)
        for row in range(BOARD_SIZE)
        for col in range(BOARD_SIZE)
        if board_movable(board, row, col, BOARD_SENTE)
    ]
    assert len(moves) == 4

def test_move_flips_stone():
    board = initial_board()
    board_move(board, 2, 3, BOARD_SENTE)
```

```
    assert board[2][3] == BOARD_SENTE
    assert board[3][3] == BOARD_SENTE
```

これらに加えて、自分が改良した評価関数自体のテスト（極端な盤面で意図通りの符号になるか、など）も書いておくとよい。

11.4　最終課題

以下を満たす Python プログラムとレポートを提出する。

11.4.1　提出プログラム

- `reversi_yourid.py`（メインプログラム）
- `test_reversi.py`（テスト）

要件:

- 第 6 章〜第 10 章の例題・練習で作成した関数を含むこと。
- 自作の評価関数・補助関数には、何を評価しているか・どのような意図で使うかが分かるコメントを付けること。
- `python reversi_yourid.py` で人間対コンピュータ・コンピュータ対コンピュータのいずれかが実行できること。
- `python -m pytest` がエラーなく通ること。

11.4.2　レポートに含める内容

- プログラムの構成（どのファイルにどの関数があるか、関数の依存関係）
- 実装した探索法（minimax / α β）の説明
- 評価関数の設計意図（各 `board_eval2` 〜 `board_eval5` を何のために入れているか、重みをどう決めたか）
- 工夫した点（独自の改良、実験のしかたなど）
- 実験条件と対戦結果（先述の表）
- 結果に対する考察
- Git を使った開発履歴の説明（`git log --oneline` を貼る）
- 感想

11.4.3　評価の観点

- 単に勝敗だけでなく、評価関数の**設計意図**が説明されているか。

- **実験の妥当性**（同じ条件で比較しているか、対戦が一回限りで結論づけていないか）。
- コードの読みやすさ（命名、分割、コメント）。
- テストの有無と内容。
- Git の使い方（粒度、コミットメッセージ）。

11.5　改良アイデアの種

時間に余裕があれば、次のような発展も検討してほしい。

- **可動性差の評価**：`board_eval3` を「自分の合法手の点数」だけでなく「相手の合法手の点数を引く」形に拡張する。
- **辺の安定石**：`kakutei_scan` を辺方向に拡張し、辺全体の安定石を評価する。
- **空きマス数による境界**：`count < 25` などの手数による境界を、実際の空きマス数で判定するよう変更する。
- **iterative deepening**：時間予算を決めて、深さ 1, 2, 3, … と順に深くしていき、時間切れになったら最後に求まった手を返す。
- **move ordering**：`expand_node()` で生成した子節点を、簡易な評価で並べ替えてから α β 法に渡すと枝刈り効率が上がる。

これらは旧 LISP 版時代から多くの先輩が試してきた工夫である。**自分が試したことと、その結果**を残すことに価値がある。

> ℹ 次章への接続：手動チューニングの限界
>
> 本章では、人間が試行錯誤して重みを変える形で評価関数を改良した。しかし、重みの組み合わせは膨大であり、人間がすべてを試すのは現実的ではない。第 11 章では、この重み探索を**自動化する**メタヒューリスティクス（山登り法・焼きなまし法・GA）を導入する。さらに第 12 章では、評価関数自体を**データから学ぶ**機械学習的アプローチを扱う。

第 12 章

メタヒューリスティクスによる評価関数の最適化

第 9 章で実装した評価関数 `eval_node()` には、`board_eval2`〜`board_eval5` の重み（`omomi`）と `BOARD_POINT` という多くのパラメータが登場した。これらの値はすべて**人間が直感で決めた**ものであり、本当に最適とは限らない。

本章では、対戦の勝率を「**適合度（fitness）**」として、評価関数の重みを**自動的に最適化**することを試みる。これは AI の歴史で言えば「**人間が知識を作り込む**」段階から「**最適化アルゴリズムが知識を獲得する**」段階への移行であり、本実験では**メタヒューリスティクス**と呼ばれる一群の探索アルゴリズムを用いる。

> **第 12 章（学習）との違い**
>
> 本章で扱うのは、ゲームのルールや評価関数の構造はそのままにして、**重みだけを自動探索する**最適化である。第 12 章では、対戦の局面と勝敗のデータから**評価関数自体を学習**する機械学習的アプローチを扱う。両者は補完的な関係にあるが、出発点と「学ぶ対象」が異なる。

12.1 本章の到達目標

- 評価関数の重みを最適化対象とした**ブラックボックス最適化問題**として定式化できる。
- **山登り法、焼きなまし法、遺伝的アルゴリズム**を実装し、同じ問題に対する挙動を比較できる。
- 各アルゴリズムの探索の特性（局所性／大域性、収束速度、計算予算）を実験的に比較できる。

12.2　問題の定式化

評価関数の重みベクトルを次のように書く。

$$\mathbf{w} = (w_2, w_3, w_4, w_5)$$

ここで w_2, w_3, w_4, w_5 はそれぞれ `board_eval2`〜`board_eval5` の `omomi` に相当する。求めたいのは、

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \ \mathrm{fitness}(\mathbf{w})$$

である。ここで $\mathrm{fitness}(\mathbf{w})$ は、重み $\mathbf{w}$ を持つ AI を**ベースライン AI**（人間が決めた基準重みを持つ AI）と複数回対戦させたときの**勝率**などとする。

> ⚠ 評価のコストに注意
>
> $\mathrm{fitness}(\mathbf{w})$ を 1 回計算するには、**ゲームを N 回プレイ**する必要がある。1 ゲームに数秒かかるため、評価呼び出し回数を抑える工夫（探索深さを浅くする、N を小さく取る、4×4 盤に縮小する）が現実的になる。

12.3　共通フレームワーク

12.3.1　例題：パラメタライズした評価関数

`eval_node()` を、重みベクトルを引数にとる**ファクトリ関数**として書き直す。

```
def make_eval_node(weights):
    """重みベクトルから eval_node 関数を作成して返す。"""
    w2, w3, w4, w5 = weights

    def eval_node(node, turn, count):
        board = node.board
        if count < 25:
            return (board_eval2(board, w2)
                    + board_eval3(board, turn, w3)
                    + board_eval4(board, w4)
                    + board_eval5(board, turn, w5))
        elif count < 40:
```

```
            return (board_eval2(board, w2)
                    + board_eval3(board, turn, w3 * 2)
                    + board_eval4(board, w4))
        elif count < 60:
            return (board_eval2(board, w2)
                    + board_eval3(board, turn, w3)
                    + board_eval4(board, w4))
        else:
            return board_eval(board)

    return eval_node
```

reversi_yourid.py の eval_node を直接書き換えるのではなく、モジュールグローバルとして登録する形にすると差し替えが楽になる。

```
# モジュール全体で使う「現在の評価関数」を保持する
_current_eval_node = None

def set_eval_node(fn):
    global _current_eval_node
    _current_eval_node = fn

def eval_node(node, turn, count):
    return _current_eval_node(node, turn, count)
```

12.3.2　練習 33：適合度関数を実装する

重みベクトル weights と、対戦相手のベースライン重み baseline を受け取り、n_games 回対戦して勝利数を返す関数 fitness(weights, baseline, n_games, depth) を定義せよ。

```
def play_one_game(eval_a, eval_b, depth):
    """eval_a を先手、eval_b を後手として 1 ゲーム対戦し、勝者を返す。"""
    ...
    return winner   # BOARD_SENTE / BOARD_GOTE / 0(=draw)


def fitness(weights, baseline, n_games, depth):
    eval_w = make_eval_node(weights)
```

```
    eval_b = make_eval_node(baseline)
    wins = 0
    # 公平のため、先手・後手を交代しながら半分ずつ対戦する
    for i in range(n_games):
        if i % 2 == 0:
            winner = play_one_game(eval_w, eval_b, depth)
            if winner == BOARD_SENTE:
                wins += 1
        else:
            winner = play_one_game(eval_b, eval_w, depth)
            if winner == BOARD_GOTE:
                wins += 1
    return wins
```

注意：minimax/αβ は決定的

同じ重みで同じ初期盤面なら、minimax / αβ は常に同じ手を指す。そのため対戦結果も毎回同じになり、`n_games` を増やしても情報が増えない。これを避けるには、

- 各ゲームの最初の数手をランダムに打つ。
- 子節点の探索順序にランダム性を入れる（`random.shuffle(children)` を `expand_node` の末尾に入れる）。
- 評価値に小さな乱数を加えて同点を崩す。

のいずれかを採用する。本実験ではいずれの方法でもよい。

12.3.3　探索コスト削減のヒント

- **深さは浅く**：`depth = 2` でも、評価関数の差は十分に勝敗に現れる。
- **盤面を小さく**：`BOARD_SIZE = 6` に縮小するとゲームが速い。
- **対戦数は控えめに**：練習段階では `n_games = 4〜8` 程度から始める。

12.4　山登り法（Hill Climbing）

最も素朴な局所探索アルゴリズムである。現在の解の近傍をランダムに 1 つ生成し、適合度が改善するなら採用、そうでなければ捨てる、を繰り返す。

入力：初期解 `w0`、最大反復回数 `T`、ステップサイズ σ
出力：最終解

```
w ← w0
f ← fitness(w)
for t in 1..T:
    w' ← w + N(0, σ²)            # ガウス雑音による近傍生成
    f' ← fitness(w')
    if f' > f:
        w ← w'
        f ← f'
return w
```

12.4.1　練習 34：山登り法を実装する

```
import random

def hill_climbing(initial_weights, max_iter, sigma, baseline, n_games, depth):
    current = list(initial_weights)
    current_fitness = fitness(current, baseline, n_games, depth)
    history = [(0, current[:], current_fitness)]

    for t in range(1, max_iter + 1):
        neighbor = [w + random.gauss(0, sigma) for w in current]
        neighbor = [max(0.0, w) for w in neighbor]   # 重みは非負に制限
        neighbor_fitness = fitness(neighbor, baseline, n_games, depth)
        if neighbor_fitness > current_fitness:
            current = neighbor
            current_fitness = neighbor_fitness
        history.append((t, current[:], current_fitness))

    return current, history
```

12.4.2　練習 35：1 次元から動作確認

最初は次元を絞って、確実に挙動を確認する。

```
# まず w4（確定石）だけを動かす
def fitness_w4(w4, baseline, n_games, depth):
    weights = list(baseline)
    weights[2] = w4   # baseline の他の重みは固定、w4 だけ可変
```

```
    return fitness(weights, baseline, n_games, depth)
```

w4 を 0, 50, 100, 200, 500, 1000 と振って、適合度（勝利数）をプロットせよ。山型・単調な形・ぎざぎざのいずれが見られるかを観察する。

12.4.3　練習 36：4 次元での山登り

すべての重みを動かす本格的な山登り法を実行し、history の適合度の推移をプロットせよ。

考察:

- 局所最適に陥った（途中で適合度の改善が止まった）か。
- ステップサイズ σ を変えると挙動はどう変わるか。

12.5　焼きなまし法（Simulated Annealing）

山登り法は局所最適に陥る欠点がある。焼きなまし法は、**改善しなくても確率的に受け入れる**ことで局所解を脱出できる。受け入れ確率は「温度」T により制御し、T は反復とともに小さくなる（**冷却**）。

```
受け入れ確率: P = exp((f' - f) / T)
ただし f' > f なら必ず採用、f' <= f なら確率 P で採用
```

12.5.1　練習 37：焼きなまし法を実装する

```
import math

def simulated_annealing(initial_weights, max_iter, sigma, T0, T_end,
                        baseline, n_games, depth):
    current = list(initial_weights)
    current_fitness = fitness(current, baseline, n_games, depth)
    best, best_fitness = current[:], current_fitness
    history = [(0, current[:], current_fitness)]

    for t in range(1, max_iter + 1):
        T = T0 * (T_end / T0) ** (t / max_iter)   # 指数冷却
        neighbor = [max(0.0, w + random.gauss(0, sigma)) for w in current]
        nf = fitness(neighbor, baseline, n_games, depth)
```

```
        if nf > current_fitness:
            current, current_fitness = neighbor, nf
        else:
            if random.random() < math.exp((nf - current_fitness) / max(T, 1e-9)):
                current, current_fitness = neighbor, nf
        if current_fitness > best_fitness:
            best, best_fitness = current[:], current_fitness
        history.append((t, current[:], current_fitness))

    return best, history
```

12.5.2　練習 38：温度スケジュールの効果

T0、T_end を変えて結果を比較せよ。T0 が大きすぎるとランダムウォーク、T0 が小さすぎると山登り法と同じ挙動になる。

12.6　遺伝的アルゴリズム（Genetic Algorithm, GA）

集団ベースの大域探索手法である。複数の重みベクトル（**個体**）を保持し、適合度の高いものを選んで**交叉**と**突然変異**で次世代を作る。

```
入力: 個体数 N、世代数 G、交叉確率 pc、突然変異確率 pm
出力: 最終世代の最良個体

集団 P ← ランダムに N 個の個体を生成
for g in 1..G:
    各個体の適合度を計算
    新集団 P' ← []
    while |P'| < N:
        親 a, b ← トーナメント選択(P)
        子 c, d ← 交叉(a, b)（確率 pc）
        c, d ← 突然変異(c, d)（確率 pm）
        P' に c, d を追加
    P ← P'
return P の最良個体
```

12.6.1　練習 39：トーナメント選択

```
def tournament_select(population, fitnesses, k=3):
    """k 個をランダムに選び、その中で最良の個体を返す"""
    candidates = random.sample(range(len(population)), k)
    best = max(candidates, key=lambda i: fitnesses[i])
    return population[best]
```

12.6.2　練習 40：交叉と突然変異

```
def crossover(a, b):
    """一様交叉：各次元について確率 0.5 で親を入れ替える"""
    c = [a[i] if random.random() < 0.5 else b[i] for i in range(len(a))]
    d = [b[i] if random.random() < 0.5 else a[i] for i in range(len(a))]
    return c, d


def mutate(individual, sigma, pm):
    """各次元に確率 pm でガウス雑音を加える"""
    return [
        max(0.0, w + random.gauss(0, sigma)) if random.random() < pm else w
        for w in individual
    ]
```

12.6.3　練習 41：GA 本体

```
def genetic_algorithm(pop_size, generations, baseline, n_games, depth,
                      sigma=20.0, pc=0.8, pm=0.2):
    # 初期集団：baseline まわりの乱数で初期化
    population = [
        [max(0.0, w + random.gauss(0, sigma * 2)) for w in baseline]
        for _ in range(pop_size)
    ]
    history = []

    for g in range(generations):
```

```
        fitnesses = [fitness(ind, baseline, n_games, depth) for ind in population]
        best_idx = max(range(pop_size), key=lambda i: fitnesses[i])
        history.append((g, population[best_idx][:], fitnesses[best_idx]))

        new_pop = [population[best_idx][:]]   # エリート保存
        while len(new_pop) < pop_size:
            a = tournament_select(population, fitnesses)
            b = tournament_select(population, fitnesses)
            if random.random() < pc:
                c, d = crossover(a, b)
            else:
                c, d = a[:], b[:]
            c = mutate(c, sigma, pm)
            d = mutate(d, sigma, pm)
            new_pop.append(c)
            if len(new_pop) < pop_size:
                new_pop.append(d)
        population = new_pop

    fitnesses = [fitness(ind, baseline, n_games, depth) for ind in population]
    best_idx = max(range(pop_size), key=lambda i: fitnesses[i])
    return population[best_idx], history
```

計算予算の見積り

1 世代の評価コストは「個体数 × 1 個体の対戦数」回のゲーム実行になる。個体数 10、世代数 10、1 個体あたり 4 ゲームなら、合計 400 ゲーム。1 ゲームが 2 秒なら 800 秒（約 13 分）。最初は個体数 6、世代数 6、対戦数 2 程度で動作確認するとよい。

12.7　アルゴリズム比較実験

12.7.1　練習 42：適合度推移の比較

山登り法、焼きなまし法、GA を**同じ評価予算**（同じ総ゲーム数）で実行し、最終的な最良適合度をそれぞれ記録する。`history` から適合度の推移をプロットすると、各アルゴリズムの特性が見える。

アルゴリズム	評価予算（ゲーム数）	最終最良適合度	最良重み
Hill Climbing	200		
Simulated Annealing	200		
Genetic Algorithm	200		

12.7.2　練習 43：考察

次の点を考察せよ。

- 同じ評価予算で、どのアルゴリズムが最も良い解を見つけたか。
- 局所最適に陥りやすいのはどれか。
- 集団ベース（GA）の利点と欠点。
- 評価のノイズ（ゲームの確率性）がアルゴリズムに与える影響。
- ベースラインに勝つ重みは見つかったか。見つかった場合、それは過学習（特定の対戦相手だけに勝つ）になっていないか。

12.8　発展：他のアルゴリズム

時間に余裕があれば、次のアルゴリズムも比較対象にできる。

- **PSO（Particle Swarm Optimization）**：粒子の速度を「個人最良」「群れ最良」へ向けて更新する。連続値最適化に向く。
- **CMA-ES**：分布の共分散行列を適応的に更新する高性能な進化戦略。`pip install cma` で使える。
- **ACO（Ant Colony Optimization）**：本来は組合せ最適化向けの手法。本問題に適用するなら、**評価項目（特徴）の部分集合**を蟻に選ばせる「**特徴選択型 ACO**」が自然である。`board_eval2`〜`board_eval5` のうちどれを使うかを決める問題に置き換える。

12.8.1　ACO の特徴選択への応用（オプション）

```
特徴の集合 F = {f2, f3, f4, f5, ...}
各特徴 f に対するフェロモン量 τ(f)
各蟻は確率 P(f) ∝ τ(f)^α で特徴を選ぶ
選んだ部分集合で AI を構成し、勝利数を測る
良かった蟻のフェロモンを増やす（更新）
悪かった蟻のフェロモンは蒸発で減る
```

このように、ACO は本来「離散的な選択を多数組み合わせる」問題に向いた手法であることを実装を通じて体験できる。

12.9　まとめ

- 評価関数の重みを**自動的にチューニングする**最適化問題として定式化することで、人間が決めた重みを超えられる可能性がある。
- 山登り法はベースラインだが局所最適に弱い。焼きなまし法は確率的に脱出できる。GA は集団で同時に複数の領域を探索できる。
- どのアルゴリズムも**評価コストが高い**（毎回ゲームを走らせる）ため、評価予算と探索効率のバランスが重要。
- メタヒューリスティクスは「**人間の知識**」と「**機械学習**」の中間に位置する考え方であり、両者を結ぶ重要な技法である。

第 13 章

発展：機械学習による評価関数の構築

第 11 章では、評価関数の**重み**をメタヒューリスティクスで最適化した。本章ではさらに踏み込み、評価関数**そのもの**を**データから学習**するアプローチを紹介する。これは現代の AI 研究の中心となっている**機械学習（machine learning）**の入門にもなる。

> 第 11 章との違い

章	探索する対象	入力	出力
第 11 章 メタヒューリスティクス	既知の評価関数の**重み** $\mathbf{w}$	対戦の勝率	最適な $\mathbf{w}$
第 12 章 機械学習	評価関数の**関数形そのもの**（モデルパラメータ）	局面と勝敗のデータ	局面 → 評価値の写像

メタヒューリスティクスは「人間が組んだ式の係数」を最適化するのに対し、機械学習は「式の形」もある程度データから決める。

本章は**発展課題**である。本実験の必修範囲を完成させ、`pytest` で動作を確認してから取り組むこと。

13.1 本章の到達目標

- 自己対戦で**学習用データセット**を作成できる。

- 線形回帰またはロジスティック回帰で**局面 →勝敗**の関係を学習できる。
- 学習した重みを `eval_node()` に組み込み、対戦で性能を確認できる。

13.2　学習用データの準備

13.2.1　自己対戦によるデータ収集

自己対戦の各局面に対し、その局面の特徴量と最終的な勝敗を組にして記録する。

```
import csv

def collect_self_play(n_games, depth, output_path):
    """自己対戦を行い、(features, winner) のペアを CSV に保存する。"""
    rows = []
    for g in range(n_games):
        board = initial_board()
        turn = BOARD_SENTE
        count = 1
        history = []   # この対戦中の (features, turn) の系列

        while True:
            state = board_state(board, turn)
            if isinstance(state, float):
                break
            if state == "pass":
                turn = change_turn(turn)
                continue

            features = extract_features(board)   # 後述
            history.append(features)

            n = Node(board, turn, 0, 0, 0.0)
            best = alpha_beta(n, turn, count, depth, None, None)
            board_move(board, best.row, best.col, turn)
            turn = change_turn(turn)
            count += 1

        # 勝者を全局面のラベルとして付与
        sente = board_number_check(board, BOARD_SENTE)
```

```
        gote = board_number_check(board, BOARD_GOTE)
        if sente > gote:
            label = 1
        elif sente < gote:
            label = -1
        else:
            label = 0

        for feat in history:
            rows.append(feat + [label])

    with open(output_path, "w", newline="") as f:
        csv.writer(f).writerows(rows)
```

13.2.2　練習 44：特徴量の設計

局面 board から数値ベクトルを抽出する関数 extract_features(board) を定義せよ。最初は素朴に、第 9 章で定義した評価項目をそのまま並べるとよい。

```
def extract_features(board):
    return [
        board_eval(board),
        board_eval2(board, 1),
        board_eval3(board, BOARD_SENTE, 1) - board_eval3(board, BOARD_GOTE, 1),
        board_eval4(board, 1),
        board_eval5(board, BOARD_SENTE, 1) + board_eval5(board, BOARD_GOTE, 1),
    ]
```

ここで返るリストの長さが**特徴量の次元**になる。発展課題として、自分が新たに思いついた特徴量（たとえば「角の所有数の差」「辺の安定石の数」など）を追加して次元を増やせる。

13.3　線形回帰による評価関数

最も単純なモデルは線形回帰である。特徴量ベクトル $\mathbf{x}$、重みベクトル β、バイアス b について

$$\hat{y} = \beta^\top \mathbf{x} + b$$

を「先手の勝ち 1、後手の勝ち −1、引き分け 0」のラベルにフィットさせる。

13.3.1　練習 45：scikit-learn で学習する

```
import numpy as np
from sklearn.linear_model import LinearRegression

data = np.loadtxt("self_play.csv", delimiter=",")
X = data[:, :-1]
y = data[:, -1]

model = LinearRegression()
model.fit(X, y)
print("coefficients:", model.coef_)
print("intercept:   ", model.intercept_)
```

学習された係数 `model.coef_` は、特徴量の各次元に対する重みである。

13.3.2　練習 46：学習結果を `eval_node()` に組み込む

```
class LearnedEvaluator:
    def __init__(self, coef, intercept):
        self.coef = coef
        self.intercept = intercept

    def __call__(self, node, turn, count):
        x = extract_features(node.board)
        return float(np.dot(self.coef, x) + self.intercept)


# 使用例
learned = LearnedEvaluator(model.coef_, model.intercept_)
set_eval_node(learned)    # 第 11 章で導入した差し替え機構を使う
```

第 11 章で導入した `set_eval_node` 機構をそのまま流用できる。

13.3.3　練習 47：手動評価関数との対戦

学習した評価関数と、第 9 章の手動評価関数 `eval_node` を対戦させ、勝率を比較せよ。データ量・特徴量・モデルの違いで結果がどう変わるかを観察する。

13.4　ロジスティック回帰

線形回帰は連続値を予測するモデルだが、勝ち負けは本来**分類**の問題である。ロジスティック回帰を使うと、予測値を確率として解釈できる。

```
from sklearn.linear_model import LogisticRegression

# 引き分けは除き、勝ち（1）と 負け（-1）の二値に絞る
mask = y != 0
X2, y2 = X[mask], (y[mask] > 0).astype(int)

model = LogisticRegression(max_iter=1000)
model.fit(X2, y2)
```

`predict_proba` で得られる「先手勝率」をそのまま評価値として使える。

13.5　小規模ニューラルネットワーク

特徴量に非線形な関係があると思える場合は、**多層パーセプトロン（MLP）**を使うとよい。

```
from sklearn.neural_network import MLPRegressor

model = MLPRegressor(hidden_layer_sizes=(16, 8), max_iter=2000, random_state=0)
model.fit(X, y)
```

PyTorch を使うとさらに柔軟なモデル設計ができるが、本実験の範囲では scikit-learn の MLP で十分である。

13.6　学習データの注意点

13.6.1　データの偏り

自己対戦データは、**今の AI の打ち手分布**に強く依存している。そのため、学習で得られる評価関数も「現在の AI が好む局面に最適化された」ものになる。本来未知の局面（たとえば人間との対戦で出てくる局面）には弱いままであることがある。

これを和らげる方法として、

- 異なる重みの AI を複数用意し、それらの組み合わせで自己対戦する。
- 序盤の数手をランダムにすることで、棋譜の多様性を増やす。
- 公開棋譜（オセロのプロ棋譜など）を学習に混ぜる。

などが考えられる。

13.6.2　過学習

特徴量の次元やモデルが大きすぎると、学習データの「**ノイズ**」まで覚えてしまい未知データで弱くなる。これを**過学習（overfitting）**と呼ぶ。対策としては、

- 学習データ・検証データの分割。
- 正則化（`Ridge`、`Lasso`、`LogisticRegression(C=...)` など）。
- モデルの規模を小さくする。

がある。本実験では学習データを小さく取らざるを得ないので、まずは**少ない特徴量**から始めるのが安全である。

13.7　発展：強化学習

オセロは状態と行動が明確に定義されているため、**強化学習**の題材としても扱いやすい。

- **TD 学習**：「現在の局面の評価」と「1 手進めた後の局面の評価」が一致するように、評価関数を逐次更新する。
- **AlphaZero 流のセルフプレイ**：方策ネットと価値ネットを同時に学習し、モンテカルロ木探索と組み合わせる。

これらは本実験の必修範囲を大きく超える。興味のある学生は卒業研究や個人プロジェクトとして取り組むことを推奨する。

13.8　レポートに含める内容（学習を選択した場合）

- 使用した特徴量の説明
- データ収集の方法（自己対戦の重み、対戦数、深さ）
- 用いたモデル（線形回帰／ロジスティック回帰／ MLP など）
- 学習結果（係数、損失、検証スコア）
- 学習した評価関数 vs 手動評価関数の対戦結果
- 過学習・データの偏りなどに対して行った対策
- 第 11 章のメタヒューリスティクス的アプローチとの比較

13.9　まとめ

- 評価関数を**データから学習**する場合、自己対戦で局面と勝敗のデータを集め、線形回帰や MLP で勝敗を予測する形に持ち込む。
- メタヒューリスティクスとは異なり、**特徴量の設計**と**過学習・データの偏り**への気配りが必要になる。
- 強化学習はさらに高度だが、本実験の枠組み（盤面表現・合法手判定・木探索）はそのまま土台として使える。

第 14 章

対戦実験と GUI

旧 LISP 版テキストでは、C 言語のクライアントと LISP の思考プログラムを**ファイル経由**で接続し、ロックファイルで排他制御を行いながら対戦していた。これは、当時 LISP では GUI を扱いにくかった事情と、複数言語のシステムを協調させる演習の意味があった。

2026 年度の本実験では、Python でオセロ本体と思考部分を**同じプログラム**として実装するため、旧版の C/LISP 間通信、LISP のコンパイル、`AI.c` の作成は扱わない。代わりに、Python で実装した `reversi_yourid.py` を使った対戦実験のやり方と、付属の **GUI 対戦ツール** `reversi-gui.py` の使い方を紹介する。

14.1 CUI での実行

`reversi_yourid.py` の末尾には、直接実行されたときだけゲームを開始する処理を書く。

```
if __name__ == "__main__":
    othello(BOARD_EMPTY, 3)
```

この例では、先手も後手もコンピュータが担当し、探索深さ 3 で実行される。人間が先手、コンピュータが後手を担当する場合は、次のように変更する。

```
if __name__ == "__main__":
    othello(BOARD_GOTE, 5)
```

ターミナルで実行する。

```
python reversi_yourid.py
```

14.2　対戦ログの保存

対戦結果をレポートに使うため、実行結果をファイルに保存する。

```
python reversi_yourid.py > log_depth3.txt
```

人間が着手する場合は対話的に動くため、画面を見ながら必要な部分を記録してよい。コンピュータ同士の対戦では、ログを保存しておくと後で比較しやすい。

14.3　条件を変えた比較

評価関数や探索深さを変更するときは、条件を明確にして比較する。

```
othello(BOARD_EMPTY, 2)
othello(BOARD_EMPTY, 3)
othello(BOARD_EMPTY, 4)
```

> **Note**
>
> 探索深さを大きくすると**必ず強くなるとは限らない**。評価関数が不適切な場合、深く読んでも誤った局面を高く評価してしまうことがある。評価関数と探索深さは**組み合わせて**評価する必要がある。

14.4　GUI による対戦：`reversi-gui.py`

`codes/reversi-gui.py` には、`pygame` を使ったオセロの GUI が用意されている。人間対 AI、人間対人間、AI 対 AI の 3 通りで遊べる。

14.4.1　起動方法

`reversi-gui.py` は、同じディレクトリにある `reversi.py` を `import` して使う。学生が改良した `reversi_yourid.py` で動かすには、いくつか方法がある。

1. **シンボリックリンク**を貼って `reversi.py` として読ませる。

   ```
   ln -s reversi_yourid.py reversi.py
   python reversi-gui.py
   ```

2. `reversi-gui.py` の冒頭の `import reversi as rv` を `import reversi_yourid as rv` に書き換える。

Dev Container での pygame 表示

Docker コンテナ内で pygame のウィンドウを表示するには、X サーバや WSLg、もしくは VSCode の Remote Display 拡張のような仕組みが必要になる。表示できない環境では、CUI 版（`reversi_yourid.py`）で動作確認してから、ホスト OS 上で GUI を起動する形でも構わない。

14.4.2　操作

1. 起動すると、先手と後手をそれぞれ「HUMAN」「AI」のどちらにするかを選ぶ画面が表示される。
2. ボタンを押して切り替え、START を押すとゲーム開始。
3. 人間の番では、ボードのマスをクリックして石を置く。
4. AI の番では、自動で `alpha_beta` を呼んで手を決める。
5. ゲームが終わると、勝敗とスコアが下部に表示される。

14.4.3　内部構造

`reversi-gui.py` は、本書で実装してきた次の関数をそのまま使っている。

- `rv.initial_board()`、`rv.board_movable()`、`rv.board_move()`：ルール部分。
- `rv.board_state()`、`rv.board_number_check()`：終局判定とスコア。
- `rv.Node`、`rv.alpha_beta()`：思考部分。

つまり、本書で作る関数のインタフェース（引数と戻り値）を保っている限り、改良した評価関数や探索手法でそのまま GUI 対戦できる。**最終課題で自分の AI を強化したら、ぜひ GUI 上で人間と対戦してみてほしい。**

14.5　Git で実験を管理する

評価関数を試すときは、変更ごとにコミットしておく。

```
git status
git add reversi_yourid.py
git commit -m "adjust corner weights"
```

うまくいかなかった変更も、結果を記録しておくとレポートで説明しやすい。不要になった実験コードを残す場合は、関数名やコメントで目的が分かるようにする。

ブランチを使うと、実験ごとに枝分かれを作って後で比較できる。

```
git switch -c experiment-stronger-corner    # 新しいブランチで実験
# ... コードを変更 ...
git commit -m "increase corner weight to 200"
# 良ければ main にマージ、悪ければブランチを破棄
```

14.6　提出前の確認

提出前に次を確認する。

- `python reversi_yourid.py` でエラーなく実行できる。
- 先手・後手・コンピュータ同士のいずれかで最後まで対戦できる。
- `python -m pytest` がエラーなく通る。
- 最終的に使っている評価関数がレポートで説明されている。
- Git のコミット履歴から、段階的に開発したことが分かる。

14.7　発展：外部対戦サーバとの接続

外部の対戦サーバや自動採点器に接続する場合は、授業で指定された入出力仕様に合わせて、`reversi_yourid.py` の上に**薄いインタフェース層**を追加する。本書で作成した合法手判定・盤面更新・探索・評価関数はそのまま再利用できる。

旧 LISP 版で扱われていたファイル経由のプロトコル（`/tmp/ban.iXXXXX.saki` などのデータファイルとロックファイル）も、Python であれば `pathlib` と `time.sleep` を使って素直に書ける。ただし、本年度の必修範囲ではここまで踏み込まないので、興味のある学生向けの発展課題として位置付ける。

14.8　まとめ

- 対戦実験は条件をそろえて行い、ログを保存する。
- `reversi-gui.py` は本書で作成した関数をそのまま使う pygame ベースの GUI である。インタフェースを保っていれば自分の改良 AI でそのまま動く。
- 実験ごとに `git commit` し、開発履歴を残すことが最終評価で重要になる。

第 15 章

まとめと評価

本実験の評価では、**完成したプログラム**、**レポート**、**口頭説明**を総合して判断する。ここでは提出物・レポートの観点・口頭説明の観点を整理する。

15.1 提出物

提出物は次の通りである。

- `reversi_yourid.py`：本書の例題・練習で作成したすべての関数を含む単一の Python ファイル。
- `test_reversi.py`：自分が書いた単体テスト。
- 最終レポート：実験条件、結果、考察を記したもの。
- 対戦ログ・実験結果：レポートに引用したログファイル。
- （任意）独自に作成したスクリプトやノートブック。

`reversi_yourid.py` には、

- 第 6 章で実装した盤面のルール部分。
- 第 7 章で実装した minimax 法。
- 第 8 章で実装した α β 法。
- 第 9〜10 章で実装した評価関数。

を含めること。独自に追加した関数には、何を評価しているのか、どのような意図で使うのかが分かる**コメントまたは docstring** を付ける。

15.2 レポートの観点

レポートでは次の内容を説明する。

- オセロプログラム全体の構成（モジュール、関数の依存関係）。
- 合法手判定と石の反転の実装方法。
- minimax 法の実装方法。
- α β 枝刈りの実装方法と minimax 法との節点数の比較。
- 評価関数の設計意図（各 `board_evalN` を何のために入れているか、重みをどう決めたか）。
- 改良前後の実験結果（条件の表、対戦結果、勝敗）。
- テストや Git をどのように使ったか（`pytest` の出力、`git log --oneline` の貼り付け）。
- 考察（うまくいったこと、うまくいかなかったこと、次にやるとしたら何をするか）。

レポートのフォーマットは Markdown / PDF / Word のいずれでも構わないが、**コードブロックと図表を読みやすく**配置すること。

15.3　口頭説明

口頭説明（プレゼンテーションや個別質疑）では、主に次の点を確認する。

- 自分のプログラムの主要関数を**コードを見ながら**説明できるか。
- 評価関数の工夫を、**なぜそうしたか**まで説明できるか。
- 実験結果に基づいて、何が良くなり、何が課題として残ったかを説明できるか。
- エラーや不具合に対して、どのように調査し、どのように直したかを説明できるか。

評価では、勝敗だけでなく、

- 実装の正しさ
- 説明の明確さ
- 開発過程の記録（Git ログ、テスト）
- 実験計画と考察の妥当性

を重視する。派手な手法を採用していなくても、**自分が考えて改良し、実験で確かめた**ことが伝われば十分に高評価となる。

15.4　評価の重み（目安）

観点	重み
基本機能の完成度（第 6〜8 章）	40%
評価関数の設計と実験（第 9〜10 章）	25%
Git・テストの活用（第 3〜4 章）	10%
レポート・口頭説明の質	20%

観点	重み
発展課題（第 11〜12 章のいずれか）への取り組み	+5%（加点）

第 11 章（メタヒューリスティクスによる最適化）または第 12 章（機械学習）への取り組みは加点対象である。着手のみでも記録があれば評価する。最終的にベースラインに勝てなくても、**何を試し、結果がどうだったか**を記録できていれば評価する。

15.5　学びの振り返り

最後に、本実験全体を通じて意識してほしい点をまとめる。

- 大きなシステムは**分割して作り、テストで動作を保証しながら積み上げる**。
- うまくいった状態を `git commit` で残しておくと、失敗からの復帰が容易になる。
- 評価関数の改良は**実験で確かめる**。一度の対戦で結論を出さない。
- 強い AI を作ることそのものよりも、**何を試し、どう判断したか**が知能工学実験の中心である。

第 16 章

参考文献

渡辺宙志. 2020. *GitHub* **演習**. https://github.com/kaityo256/github.